FATEC SÃO PAULO

Faculdade de Tecnologia de São Paulo

Centro Estadual de Educação Tecnológica Paula Souza

Guia Prático

Etapa 2: Power Query

Projeto Evolutivo de Dados – Painel de Performance da Rede CaféData Brasil
Microinformática Aplicada – Fatec São Paulo – 2026/2

Roteiro operacional para ser seguido **com o Excel e o Editor do Power Query abertos**. Ao final, você terá o arquivo `CafeData_02_PowerQuery.xlsx`, salvo em `MA2026-entrega-parcial/02-powerquery/` no repositório da equipe, com as consultas `fVendas`, `dProduto`, `dFilial`, `fMetas` e `Rejeitados`, conforme a Seção 6.2 da especificação oficial *Projeto Modelo – Painel de Performance da Rede CaféData Brasil* (versão 2.0, outubro de 2026). Cada procedimento traz objetivo, pré-requisitos, passos, código M exato, resultado esperado, forma de conferência e erros comuns.

Prof. Veríssimo

São Paulo, outubro de 2026

Sumário

1 Como usar este guia **4**
1.1 O que entra e o que sai . . . 4
1.2 Escopo: o que é e o que não é da Etapa 2 . . . 4
1.3 Mapa das consultas . . . 5
1.4 Requisitos do computador . . . 5
1.5 Convenções deste guia . . . 6

2 Atividade 1 – Preparar o ambiente e conectar à pasta de vendas **6**
2.1 Organizar pastas, arquivos de entrada e repositório . . . 6
2.2 Configurar o Power Query: localidade, privacidade e parâmetros . . . 8
2.3 Conectar aos cadastros da Etapa 1 . . . 9
2.4 Conectar à pasta de vendas . . . 11

3 Atividade 2 – Unificar e consolidar as filiais (anexar consultas) **12**
3.1 Diagnosticar as diferenças de colunas entre filiais (D10) . . . 12
3.2 Criar o de-para de colunas e a função de padronização . . . 13
3.3 Anexar as filiais em uma única consulta . . . 15
3.4 Conferência das colunas por arquivo . . . 16

4 Atividade 3 – Limpeza de dados, nulos e duplicidades **17**
4.1 Criar a consulta fVendas e remover linhas em branco . . . 17
4.2 Tratar nulos nas colunas críticas e não críticas (D5) . . . 18
4.3 Remover duplicidades pela chave da transação (D8) . . . 20

5 Atividade 4 – Ajuste de tipos e localidade pt-BR **20**
5.1 Datas e horas em formatos diferentes (D1) . . . 21
5.2 Números com vírgula guardados como texto (D6) . . . 23

6 Atividade 5 – Padronização de textos e maiúsculas **24**

7 Atividade 6 – Mesclagem com o De-Para de Filiais **26**
7.1 Obter o código oficial da filial (D4) . . . 26
7.2 Obter o código do produto no cadastro (D7) . . . 27
7.3 Segunda passagem de duplicidades pela chave oficial (D8) . . . 28

8 Atividade 7 – Criação de colunas personalizadas **29**
8.1 Contagem de linhas etapa por etapa . . . 30

9 Atividade 8 – Tratamento de exceções e consulta de rejeitados **31**

9.1 Criar a consulta Rejeitados 31
9.2 Consultas de conferência 33

10 Atividade 9 – Carga dos dados tratados para o Excel 34
10.1 Consulta fMetas: despivotar as metas mensais 34
10.2 Carregar as consultas em planilhas 36
10.3 Publicar no repositório e revisar a entrega 37

A Código M completo da consulta fVendas 38

B Funções e trechos complementares 40

C Filial que envia CSV em vez de planilha 40

D Gabarito completo do exemplo simulado 41

E Quando a base oficial for liberada: o que substituir 42

F Transposição para o tema da equipe 42

1. Como usar este guia

1.1 O que entra e o que sai

A Etapa 1 transformou **uma** planilha de filial em uma base auditável, com fórmulas. A Etapa 2 faz o mesmo trabalho de forma **automática e repetível** para **todas** as filiais de uma vez: o Power Query lê a pasta inteira de arquivos brutos, aplica as regras de limpeza e entrega uma única tabela tratada. Quando chegar um arquivo novo ou corrigido, basta clicar em **Atualizar Tudo**.

Entrada	Uso nesta etapa	Local
Planilhas brutas das filiais	Fonte principal: todos os arquivos `.xlsx` da pasta, lidos e anexados automaticamente.	`C:\CafeData\dados_brutos\vendas\`
`CafeData_01_Excel.xlsx`	Saída da Etapa 1. Fornece `Tab_Produtos`, `Tab_Filiais`, `Tab_DePara_Filial`, `Tab_Pagamento` e `Tab_Canal`.	`C:\CafeData\01_excel\`
`metas_mensais.xlsx`	Origem de **`fMetas`** (meta por filial e mês).	`C:\CafeData\dados_brutos\`
Aba **`Auditoria`** da Etapa 1	Base conceitual: as regras que lá eram *sinalizadas* viram aqui *filtros* com registro em **`Rejeitados`**.	(consulta, não é lida pelo Power Query)

Saída	Conteúdo	Local
`CafeData_02_PowerQuery.xlsx`	Pasta de trabalho com as consultas e as tabelas carregadas: **`fVendas`**, **`Rejeitados`**, **`dProduto`**, **`dFilial`**, **`fMetas`** e **`Conferencia`**.	`MA2026-entrega-parcial/02-powerquery/`
`scripts_m/*.pq`	Código M de cada consulta, copiado do Editor Avançado (um arquivo de texto por consulta).	`02-powerquery/scripts_m/`
`dados_limpos/fVendas.csv`	Cópia dos dados tratados em CSV (UTF-8), exigida pelo Guia de Orientação (“dados limpos”).	`02-powerquery/dados_limpos/`
`evidencias/` e `README.md`	Capturas de tela do painel de consultas e da conferência; descrição das consultas e dos parâmetros.	`02-powerquery/`

1.2 Escopo: o que é e o que não é da Etapa 2

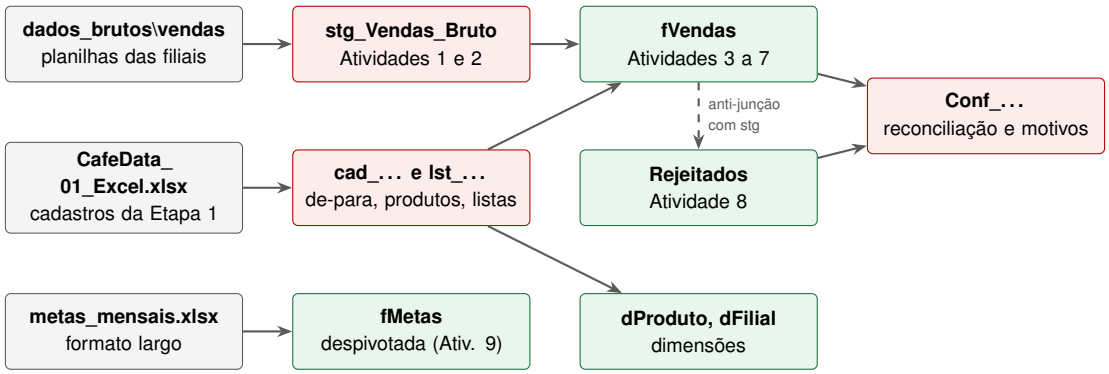
A tabela abaixo retoma os defeitos propositais da especificação (Seção 4.3) e diz como cada um é tratado **nesta** etapa. Power Pivot (relacionamentos e medidas DAX), VBA, SQL, APIs e Power BI ficam para as etapas seguintes.

#	Defeito	Na Etapa 2	Onde no guia
D1	Datas em formatos diferentes	**Tratar**	Função `fnData`; data inexistente vai para `Rejeitados` (Seção 5.1)
D2	Categoria sem padrão de caixa	**Tratar**	`Text.Proper` (Seção 6)
D3	Espaços extras nos nomes	**Tratar**	`Text.Trim` (Seção 6)
D4	Nome de filial com variações	**Tratar**	Mesclagem com o de-para de filiais (Seção 7)
D5	Quantidade vazia ou zero	**Filtrar e registrar**	Filtro de nulos; a linha vai para `Rejeitados` (Seção 4.2)
D6	Valor como texto com vírgula	**Tratar**	Tipo de dado com localidade pt-BR (Seção 5)
D7	Produto fora do cadastro	**Filtrar e registrar**	Mesclagem com o cadastro; sem código, a linha vai para `Rejeitados` (Seção 7)
D8	Linhas duplicadas	**Tratar**	`Table.Distinct` pela chave da transação (Seções 4.3 e 7.3)
D9	Preço muito acima/abaixo da ref.	**Não tratar**	Não é motivo de descarte: a venda existe. Continua registrado na auditoria da Etapa 1
D10	Colunas diferentes entre filiais	**Tratar**	De-para de colunas e função `fnPadronizarColunas` (Atividade 3)

Atenção

Regra de ouro da etapa: nenhuma linha some sem explicação. Toda linha lida dos arquivos brutos termina **ou** em `fVendas` **ou** em `Rejeitados`, com o motivo. A consulta `Conf_Reconciliacao` prova isso: *linhas lidas* − *aceitas* − *rejeitadas* = **0**. Os arquivos brutos continuam intocados: o Power Query só lê.

1.3 Mapa das consultas



Caixas cinza: arquivos de origem. Vermelhas: consultas de apoio (somente conexão). Verdes: consultas carregadas em planilhas. As funções (`fn...`) e os parâmetros (`p...`) não aparecem no desenho.

1.4 Requisitos do computador

- **Excel para Windows, Microsoft 365 ou Excel 2021 (ou mais recente).** O Power Query já vem embutido (guia **Dados** → **Obter e Transformar Dados**). No Excel para Mac e para a Web vários conectores e o Editor Avançado são limitados: use o laboratório ou um Windows.

- O arquivo `CafeData_01_Excel.xlsx` da Etapa 1 **concluído e salvo**, com as tabelas da aba `Cadastros` com os nomes exatos do guia anterior.
- Os nomes de menu seguem o Excel do Microsoft 365 em português. Em versões em inglês: **Get Data**, **Transform Data**, **Advanced Editor**, **Merge Queries**, **Append Queries**.

Atenção

A linguagem M não muda com o idioma do Excel. Diferente das fórmulas da Etapa 1 (`PROCX`, `ARRUMAR`, separador `;`), o código M é **sempre** em inglês, usa **vírgula** para separar argumentos, **ponto** como separador decimal nos números escritos no código (`0.2`) e **diferencia maiúsculas de minúsculas**: `Text.Trim` funciona, `text.trim` dá erro.

1.5 Convenções deste guia

Notação	Significado
Página Inicial → **Editor Avançado**	Guia e botão da faixa de opções **do Editor do Power Query** (salvo indicação de que é no Excel).
`fVendas`	Nome de uma consulta, como aparece no painel **Consultas** à esquerda do Editor.
`SemCupomVazio`	Nome de uma **etapa** da consulta, como aparece em **Etapas Aplicadas** (à direita).
`[Quantidade]`	Valor da coluna `Quantidade` na linha atual (dentro de `each`).
Caixa **amarela**	Dados simulados, criados só para a demonstração.
Caixa **azul**	O que deve aparecer na sua tela se o passo deu certo.
Caixa **verde**	Testes rápidos para confirmar o resultado.

Clicar ou digitar? Quase todas as etapas podem ser criadas pelos botões da faixa de opções; o Power Query escreve o código M sozinho. Este guia mostra **os dois**: o caminho de menu e o código exato da etapa. Depois de clicar, compare a **barra de fórmulas** (**Exibir** → **Barra de Fórmulas**) com o código do guia e renomeie a etapa (clique com o botão direito na etapa → **Renomear**) para o nome usado aqui. Quando uma etapa só puder ser digitada, clique no botão `fx` à esquerda da barra de fórmulas: surge uma etapa nova com `= NomeDaEtapaAnterior`; substitua pelo código do guia e pressione `Enter`.

2. Atividade 1 – Preparar o ambiente e conectar à pasta de vendas

2.1 Organizar pastas, arquivos de entrada e repositório

OBJETIVO

Garantir que a pasta de vendas contenha **somente** as planilhas de filial e que o arquivo de trabalho nasça no lugar exigido pelo repositório.

O QUE PRECISO TER ANTES DE COMEÇAR

A estrutura `C:\CafeData` da Etapa 1 e o repositório da equipe clonado no computador: `fatecsp-MA20262-equipe-nn-nome-projeto`.

PASSOS

1. Copie para `C:\CafeData\dados_brutos\vendas` as planilhas de vendas das filiais. Enquanto a base

oficial não for liberada, use os simulados: `vendas_F02_zona_sul_SIMULADO.xlsx` (o mesmo da Etapa 1) e `vendas_F01_sp_centro_SIMULADO.xlsx` (novo, acompanha este guia).

2. Copie `metas_mensais_SIMULADO.xlsx` (acompanha este guia) para `C:\CafeData\dados_brutos`.
3. Marque todos os arquivos de entrada como **Somente leitura** (botão direito → **Propriedades**), como na Etapa 1.
4. Confirme que `CafeData_01_Excel.xlsx` está em `C:\CafeData\01_excel`, salvo e **fechado**.
5. No repositório, dentro de `MA2026-entrega-parcial\02-powerquery`, crie as subpastas `scripts_m`, `dados_limpos` e `evidencias`.
6. Abra o Excel → **Pasta de trabalho em branco** → **Arquivo** → **Salvar Como** → **Procurar** → pasta `02-powerquery` do repositório → nome `CafeData_02_PowerQuery` → tipo **Pasta de Trabalho do Excel (*.xlsx)** → **Salvar**.

FÓRMULAS OU VALORES EXATOS

```
C:\CafeData\                                    (dados de entrada: o Power Query só lê)
|-- dados_brutos\
|   |-- vendas\
|   |   |-- vendas_F01_sp_centro_SIMULADO.xlsx    (novo: cabeçalhos diferentes, D10)
|   |   `-- vendas_F02_zona_sul_SIMULADO.xlsx     (o mesmo da Etapa 1)
|   `-- metas_mensais_SIMULADO.xlsx
`-- 01_excel\
    `-- CafeData_01_Excel.xlsx                    (saída da Etapa 1: cadastros)

fatecsp-MA20262-equipe-nn-nome-projeto\         (repositório da equipe)
`-- MA2026-entrega-parcial\
    `-- 02-powerquery\
        |-- CafeData_02_PowerQuery.xlsx           (arquivo que você vai criar)
        |-- scripts_m\    dados_limpos\    evidencias\
        `-- README.md
```

Atenção

A pasta de vendas não pode ter "intrusos". O Power Query vai ler **tudo** o que estiver nela: um rascunho, uma cópia `vendas_F02 - Cópia.xlsx` ou o próprio arquivo de trabalho salvo ali entram na base. Não salve nada na pasta `vendas` além das planilhas das filiais. Quando a base oficial for liberada, **retire** os arquivos `_SIMULADO` da pasta (Apêndice E).

RESULTADO ESPERADO

Pasta `vendas` com 2 arquivos (simulado) ou 5 (oficial); `CafeData_02_PowerQuery.xlsx` vazio, salvo em `02-powerquery`.

Como conferir

No Excel, **Arquivo** → **Informações**: o caminho termina em `\MA2026-entrega-parcial\02-powerquery\CafeData_02_PowerQuery.xlsx`. No Projeto Modelo (Seção 9 da especificação), a pasta equivalente da demonstração em laboratório é `CafeData\02_power_query`; para a entrega das equipes vale o repositório.

ERROS COMUNS E CORREÇÕES

Erro	Correção
Salvou o arquivo de trabalho dentro de `dados_brutos\vendas`	Mova para `02-powerquery`. Caso contrário, ele será lido como se fosse uma filial.
Repositório dentro do OneDrive com sincronização pausada	O caminho muda de computador para computador; por isso as origens serão **parâmetros** (próxima seção).

2.2 Configurar o Power Query: localidade, privacidade e parâmetros

OBJETIVO

Fazer o Power Query interpretar números e datas no padrão brasileiro e guardar os caminhos dos arquivos em **parâmetros**, para que a equipe troque a origem em um único lugar.

O QUE PRECISO TER ANTES DE COMEÇAR

`CafeData_02_PowerQuery.xlsx` aberto.

PASSOS

1. No Excel: **Dados** → **Obter Dados** → **Opções de Consulta**.
2. Em **Pasta de Trabalho Atual** → **Configurações Regionais**, escolha **Localidade: Português (Brasil)**.
3. Em **Pasta de Trabalho Atual** → **Privacidade**, marque **Ignorar os Níveis de Privacidade e possivelmente melhorar o desempenho**. Clique em **OK**. (Todas as origens são arquivos locais da própria equipe; sem isso o Power Query bloqueia a combinação de fontes, ver Erros comuns.)
4. Abra o Editor: **Dados** → **Obter Dados** → **Iniciar Editor do Power Query**.
5. **Página Inicial** → **Gerenciar Parâmetros** → **Novo Parâmetro**. Crie os três parâmetros da tabela abaixo (**Tipo: Texto**, **Valores Sugeridos: Qualquer valor**, **Obrigatório** marcado). Clique em **OK**.
6. Selecione os três parâmetros no painel **Consultas**, clique com o botão direito → **Mover para Grupo** → **Novo Grupo** → nome `01_Parametros`.

FÓRMULAS OU VALORES EXATOS

Nome	Valor atual	Uso
`pPastaVendas`	`C:\CafeData\dados_brutos\vendas\`	pasta com as planilhas das filiais
`pArquivoEtapa1`	`C:\CafeData\01_excel\CafeData_01_Excel.xlsx`	cadastros da Etapa 1
`pArquivoMetas`	`C:\CafeData\dados_brutos\metas_mensais_SIMULADO.xlsx`	metas mensais

Selecione `pPastaVendas` e abra **Página Inicial** → **Editor Avançado**. O parâmetro é uma consulta comum com metadados:

```
"C:\CafeData\dados_brutos\vendas\" meta [IsParameterQuery = true, Type = "Text",
    IsParameterQueryRequired = true]
```

RESULTADO ESPERADO

O que você deve ver na tela

Painel **Consultas** com o grupo `01_Parametros` contendo `pPastaVendas`, `pArquivoEtapa1` e `pArquivoMetas`, cada um com o ícone de parâmetro e o valor exibido ao lado.

Como conferir

Copie o valor de `pPastaVendas`, cole na barra de endereço do Explorador de Arquivos e pressione `Enter`: deve abrir a pasta com as planilhas das filiais. Faça o mesmo com os dois arquivos: devem abrir no Excel.

ERROS COMUNS E CORREÇÕES

Erro	Correção
Caminho copiado com aspas (`"C:\...\"`)	No campo **Valor Atual**, digite sem aspas: o Power Query as acrescenta.
Mais tarde: `Formula.Firewall: a consulta faz referência a outras consultas...`	Faltou o passo 3 (privacidade). Refaça e clique em **Atualizar Visualização**.
`8,50` lido como `850` mais adiante	A localidade do arquivo ficou em inglês. Refaça o passo 2 (e use sempre `"pt-BR"` nas conversões, Seção 5).

2.3 Conectar aos cadastros da Etapa 1

OBJETIVO

Reaproveitar, sem redigitar, os cadastros validados na Etapa 1: produtos, filiais, de-para de filiais e listas oficiais de pagamento e canal.

O QUE PRECISO TER ANTES DE COMEÇAR

Parâmetro `pArquivoEtapa1`; `CafeData_01_Excel.xlsx` **salvo** (o Power Query lê a última versão gravada em disco).

PASSOS

1. No Editor: **Página Inicial** → **Nova Fonte** → **Outras Fontes** → **Consulta Nula**. Abra o **Editor Avançado**, apague o conteúdo, cole o código de `cad_Etapa1` e clique em **Concluído**. No painel **Configurações de Consulta** (direita), em **Nome**, digite `cad_Etapa1`.
2. Repita, uma consulta nula por vez, para `dProduto`, `dFilial`, `cad_DeParaFilial`, `cad_ProdutoChave`, `lst_Pagamento` e `lst_Canal`.
3. Agrupe `cad_Etapa1`, `cad_DeParaFilial`, `cad_ProdutoChave` e as duas `lst_` em `03_Cadastros`; `dProduto` e `dFilial` em `05_Saida`.

CÓDIGO M EXATO

```
// cad_Etapa1 -- lista tudo o que existe no arquivo da Etapa 1 (abas, tabelas, nomes)
let
    Fonte = Excel.Workbook(File.Contents(pArquivoEtapa1), null, true)
in
    Fonte

// dProduto -- dimensão de produtos (Tab_Produtos da Etapa 1)
let
    Fonte = cad_Etapa1{[Item = "Tab_Produtos", Kind = "Table"]}[Data],
    Renomeada = Table.RenameColumns(Fonte, {{"Cod", "Cod_Produto"}}),
    Tipos = Table.TransformColumnTypes(Renomeada, {{"Cod_Produto", type text}, {"Produto", type text
    },
              {"Categoria", type text}, {"Preco_Ref", type number}, {"Custo", type number}})
in
```

```
    Tipos

// dFilial -- dimensão de filiais (Tab_Filiais da Etapa 1)
let
    Fonte = cad_Etapa1{[Item = "Tab_Filiais", Kind = "Table"]}[Data],
    Tipos = Table.TransformColumnTypes(Fonte, {{"Cod_Filial", type text}, {"Filial", type text},
             {"Cidade", type text}, {"Lat", type number}, {"Long", type number}, {"Abertura", Int64.
    Type}})
in
    Tipos

// cad_DeParaFilial -- de-para com uma chave em MAIÚSCULAS e sem espaços nas pontas
let
    Fonte = cad_Etapa1{[Item = "Tab_DePara_Filial", Kind = "Table"]}[Data],
    Tipos = Table.TransformColumnTypes(Fonte, {{"Nome_Origem", type text}, {"Cod_Filial", type text
    }}),
    ComChave = Table.AddColumn(Tipos, "Chave_Filial", each Text.Upper(Text.Trim([Nome_Origem])),
    type text)
in
    ComChave

// cad_ProdutoChave -- só o necessário para encontrar o código pelo nome
let
    ComChave = Table.AddColumn(dProduto, "Chave_Produto", each Text.Upper(Text.Trim([Produto])),
    type text),
    Selecionadas = Table.SelectColumns(ComChave, {"Chave_Produto", "Cod_Produto"})
in
    Selecionadas

// lst_Pagamento e lst_Canal -- listas (não tabelas) com os valores oficiais
cad_Etapa1{[Item = "Tab_Pagamento", Kind = "Table"]}[Data][Pagamento]
cad_Etapa1{[Item = "Tab_Canal", Kind = "Table"]}[Data][Canal]
```

Trecho	O que faz
`Excel.Workbook( File.Contents(...), null, true)`	Abre o arquivo e devolve uma tabela com um item por aba, tabela e nome definido (colunas `Name`, `Data`, `Item`, `Kind`).
`{[Item = "Tab_Produtos", Kind = "Table"]}`	Escolhe a **linha** cujo item é a tabela `Tab_Produtos`. É o mesmo código que o navegador gera quando você clica na tabela.
`[Data]`	Pega a tabela de dados guardada nessa linha.
`Text.Upper(Text.Trim(...))`	Cria a **chave de comparação**: a mesclagem do Power Query diferencia maiúsculas, acentos e espaços; a chave neutraliza caixa e espaços nas pontas.
`...[Data][Pagamento]`	Pega uma coluna como **lista**: `{"Pix", "Débito", ...}`.

RESULTADO ESPERADO

O que você deve ver na tela

`dProduto`: 18 linhas e 5 colunas, com `Preco_Ref` e `Custo` com ícone `1.2` (número decimal). `dFilial`: 5 linhas. `cad_DeParaFilial`: 8 linhas e a coluna `Chave_Filial` com `SP-ZSUL`, `ZONA SUL` etc. `lst_Pagamento`: lista com 5 itens.

Como conferir

Clique em `cad_DeParaFilial`, selecione a coluna `Chave_Filial` e use **Página Inicial** → **Remover Linhas** → **Remover Duplicatas só para olhar**: o número de linhas na barra de status tem de continuar 8. Se cair, há grafia repetida no de-para (isso multiplicaria vendas na mesclagem). **Exclua essa etapa de teste** clicando no **X** ao lado dela em **Etapas Aplicadas**.

ERROS COMUNS E CORREÇÕES

Erro	Correção
`Expression.Error: A chave não correspondeu a nenhuma linha da tabela`	O nome da tabela no arquivo da Etapa 1 é outro. Clique em `cad_Etapa1`, leia a coluna `Item` e use o nome exato.
`A coluna 'Cod' da tabela não foi encontrada`	Na Etapa 1 a coluna se chama de outro jeito. Ajuste o nome no `Table.RenameColumns`.
`DataFormat.Error: O arquivo está sendo usado`	Feche `CafeData_01_Excel.xlsx` ou salve-o; depois **Atualizar Visualização**.
Cadastro alterado na Etapa 1 não aparece	O Power Query lê o arquivo salvo. Salve a Etapa 1 e atualize.

2.4 Conectar à pasta de vendas

OBJETIVO

Listar, por consulta, todos os arquivos da pasta `vendas`, descartando o que não é planilha de filial.

O QUE PRECISO TER ANTES DE COMEÇAR

Parâmetro `pPastaVendas`.

PASSOS

1. No Excel: **Dados** → **Obter Dados** → **De Arquivo** → **Da Pasta**. Escolha `C:\CafeData\dados_brutos\vendas` → **Abrir**.
2. Na janela com a lista de arquivos, clique em **Transformar Dados**. **Não** clique em **Combinar** (veja a nota abaixo).
3. No Editor, renomeie a consulta para `stg_Vendas_Bruto`.
4. Na etapa `Fonte`, troque o caminho digitado pelo parâmetro: na barra de fórmulas, substitua `"C:\CafeData\..."` por `pPastaVendas`.
5. Filtre a coluna `Extension` para manter `.xlsx`, e a coluna `Name` com **Filtros de Texto** → **Não Começa Com** `~$`. Renomeie as etapas para `Origem` e `SoPlanilhas` e compare com o código abaixo.

Por que não usar o botão Combinar? Ele cria consultas auxiliares (*Arquivo de Exemplo*, *Transformar Arquivo*) e usa **o primeiro arquivo como molde**: as colunas de uma filial com cabeçalhos diferentes (D10) entram como colunas a mais ou somem em silêncio. Na Atividade 3 você escreve a sua própria função de leitura, que padroniza cada arquivo **antes** de anexar.

CÓDIGO M EXATO (INÍCIO DA CONSULTA STG_VENDAS_BRUTO)

```
let
    Origem = Folder.Files(pPastaVendas),
    // só planilhas .xlsx; "~$..." são arquivos temporários do Excel aberto
```

```
    SoPlanilhas = Table.SelectRows(Origem, each Text.Lower([Extension]) = ".xlsx"
                                        and not Text.StartsWith([Name], "~$"))
in
    SoPlanilhas
```

RESULTADO ESPERADO

O que você deve ver na tela

Uma linha por arquivo, com as colunas `Content` (`Binary`), `Name`, `Extension`, `Date modified` e `Folder Path`. No simulado, 2 linhas: `vendas_F01_sp_centro_SIMULADO.xlsx` e `vendas_F02_zona_sul_SIMULADO.xlsx`.

Como conferir

Abra a planilha da F02 no Excel (o arquivo fica bloqueado e aparece um `~$vendas_F02...` oculto na pasta). No Editor, clique em **Página Inicial** → **Atualizar Visualização**: a etapa `Origem` mostra 3 linhas, mas `SoPlanilhas` continua com 2. Feche a planilha.

ERROS COMUNS E CORREÇÕES

Erro	Correção
`DataSource.Error: Não foi possível encontrar uma parte do caminho`	O parâmetro aponta para pasta inexistente. Confira `pPastaVendas` (Seção 2.2).
Extensão `.XLSX` (maiúscula) não aparece	Por isso o filtro usa `Text.Lower([Extension])`.
Arquivos de subpastas aparecem	`Folder.Files` lê subpastas também. Não crie subpastas dentro de `vendas`.

3. Atividade 2 – Unificar e consolidar as filiais (anexar consultas)

3.1 Diagnosticar as diferenças de colunas entre filiais (D10)

OBJETIVO

Descobrir, arquivo por arquivo, como cada filial chama e ordena suas colunas, antes de unificar.

O QUE PRECISO TER ANTES DE COMEÇAR

Consulta **`stg_Vendas_Bruto`** até a etapa `SoPlanilhas`.

PASSOS

1. Na coluna `Content`, clique na palavra `Binary` da primeira linha. O Power Query abre o arquivo e mostra suas abas. Clique em `Table` da aba `Vendas`. Leia os cabeçalhos. **Exclua** as etapas criadas por esse clique (X em **Etapas Aplicadas**): era só inspeção.
2. Repita para o segundo arquivo.
3. Anote, numa tabela de dois campos (nome encontrado → nome-alvo), toda diferença em relação ao dicionário de dados (Seção 4.2 da especificação).

RESULTADO ESPERADO

■ EXEMPLO SIMULADO – não são dados oficiais do projeto

Pos.	F01 (SP-Centro)	F02 (Zona Sul)	Nome-alvo (dicionário)
1	`Loja`	`ID_Cupom`	`ID_Cupom`
2	`Cupom`	`Data`	`Data`
3	`Data_Venda`	`Hora`	`Hora`
4	`Hora_Venda`	`Filial`	`Filial` (vira `Cod_Filial` na Atividade 6)
5	`Item`	`Produto`	`Produto` (vira `Cod_Produto` na Atividade 6)
6	`Categoria`	`Categoria`	`Categoria`
7	`Quantidade`	`Qtd`	`Quantidade`
8	`Preço Unitário`	`Valor_Unit`	`Valor_Unit`
9	`Desconto`	`Desconto`	`Desconto`
10	`Forma Pagamento`	`Pagamento`	`Pagamento`
11	`Canal␣` (espaço no fim)	`Canal`	`Canal`

Além dos nomes, a **ordem** é diferente: na F01 a filial vem primeiro. Na base oficial, repita o diagnóstico para as cinco filiais.

Como conferir

Para cada coluna do dicionário, cada arquivo deve ter exatamente **uma** coluna correspondente. Se uma filial tiver duas candidatas (por exemplo, `Qtd` e `Quantidade`), pare e consulte o professor antes de seguir.

3.2 Criar o de-para de colunas e a função de padronização

OBJETIVO

Transformar o diagnóstico em dados (tabela `DeParaColunas`) e criar funções que leem **qualquer** planilha de filial e a entregam no formato-alvo.

O QUE PRECISO TER ANTES DE COMEÇAR

Tabela de diagnóstico da seção anterior.

PASSOS

1. Crie (**Nova Fonte** → **Outras Fontes** → **Consulta Nula** + **Editor Avançado**) as consultas `ColunasAlvo` e `DeParaColunas`.
2. Crie as funções `fnLerPlanilha`, `fnPadronizarColunas` e `fnLerVendas`. Ao clicar em **Concluído**, cada uma aparece com o ícone `fx` e uma tela para **Invocar**: **não** invoque.
3. Agrupe `ColunasAlvo` e `DeParaColunas` em `03_Cadastros`; as funções em `02_Funcoes`.

CÓDIGO M EXATO

```
// ColunasAlvo -- formato-alvo (Seção 4.2 da especificação), na ordem desejada
{"ID_Cupom", "Data", "Hora", "Filial", "Produto", "Categoria",
 "Quantidade", "Valor_Unit", "Desconto", "Pagamento", "Canal"}

// DeParaColunas -- cada nome encontrado nas filiais aponta para um nome-alvo
let
    Tabela = #table(type table [Nome_Origem = text, Nome_Alvo = text], {
        {"Qtd",          "Quantidade"},
        {"Cupom",        "ID_Cupom"},
        {"Data_Venda",   "Data"},
        {"Hora_Venda",   "Hora"},
```

```
        {"Loja",            "Filial"},
        {"Item",            "Produto"},
        {"Preço Unitário",  "Valor_Unit"},
        {"Forma Pagamento", "Pagamento"}
    })
in
    Tabela

// fnLerPlanilha -- devolve a primeira aba de um arquivo .xlsx, com cabeçalhos
(conteudo as binary) as table =>
let
    Pasta = Excel.Workbook(conteudo, true),
    Abas = Table.SelectRows(Pasta, each [Kind] = "Sheet"),
    Dados = Abas{0}[Data]
in
    Dados

// fnPadronizarColunas -- resolve o defeito D10 (nomes e ordem diferentes)
(tabela as table) as table =>
let
    // 1. tira espaços das pontas dos cabeçalhos ("Canal " -> "Canal")
    CabecalhosLimpos = Table.TransformColumnNames(tabela, Text.Trim),
    // 2. pares {nome encontrado, nome-alvo} vindos de DeParaColunas
    Pares = List.Zip({DeParaColunas[Nome_Origem], DeParaColunas[Nome_Alvo]}),
    // 3. renomeia apenas o que existir neste arquivo
    Renomeada = Table.RenameColumns(CabecalhosLimpos, Pares, MissingField.Ignore),
    // 4. reordena no formato-alvo; coluna ausente vira coluna de nulos
    NoFormatoAlvo = Table.SelectColumns(Renomeada, ColunasAlvo, MissingField.UseNull)
in
    NoFormatoAlvo

// fnLerVendas -- lê, padroniza e numera as linhas de UM arquivo de filial
(conteudo as binary) as table =>
let
    Planilha = fnLerPlanilha(conteudo),
    Padronizada = fnPadronizarColunas(Planilha),
    // Linha_Origem = número da linha na planilha original (1ª linha de dados = 2)
    ComLinha = Table.AddIndexColumn(Padronizada, "Linha_Origem", 2, 1, Int64.Type)
in
    ComLinha
```

Trecho	Por quê
`MissingField.Ignore` no renomear	Cada filial só tem *alguns* dos nomes do de-para. Sem essa opção, o primeiro nome ausente gera erro.
`MissingField.UseNull` no selecionar	Se uma filial não tiver, por exemplo, `Desconto`, a coluna é criada com nulos em vez de quebrar a consulta. Colunas *extras* são descartadas.
`Table.AddIndexColumn(..., 2, 1)`	Mesma ideia da `Linha_Origem` da Etapa 1: liga cada linha tratada à linha da planilha bruta. Começa em 2 porque a linha 1 é o cabeçalho.
De-para em **dados**, não em código	Para uma filial nova basta acrescentar uma linha em **`DeParaColunas`**. Nenhuma etapa de **`stg_Vendas_Bruto`** muda.

RESULTADO ESPERADO

O que você deve ver na tela

No grupo `02_Funcoes`: **`fnLerPlanilha`**, **`fnPadronizarColunas`** e **`fnLerVendas`**, com ícone `fx`. `ColunasAlvo` aparece como **Lista** com 11 itens; `DeParaColunas` como tabela com 8 linhas.

Como conferir

Teste a função sem usar o botão **Invocar** (ele pede um arquivo binário, que não dá para digitar). Crie uma consulta nula temporária e, na barra de fórmulas, digite `= fnLerVendas(stg_Vendas_Bruto{0}[Content])` e veja a F01 com as 11 colunas-alvo mais `Linha_Origem` (2 a 13). Exclua a consulta temporária.

ERROS COMUNS E CORREÇÕES

Erro	Correção
`Expression.Error: a coluna 'Quantidade' já existe`	O arquivo tem `Qtd` **e** `Quantidade`. Não force: registre e consulte o professor (seção anterior).
Coluna-alvo inteira com `null` em uma filial	O nome encontrado não está no de-para (ou tem acento/espaço diferente). Use a consulta `Conf_Colunas` (Seção 3.4).
`Preço Unitário` não renomeia	Diferença de acento ou espaço duplo no cabeçalho. Copie o nome do próprio Editor (duplo clique no cabeçalho → `Ctrl`+`C`).

3.3 Anexar as filiais em uma única consulta

OBJETIVO

Aplicar **`fnLerVendas`** a cada arquivo e empilhar os resultados (anexar), guardando o nome do arquivo de origem.

O QUE PRECISO TER ANTES DE COMEÇAR

Funções da seção anterior.

PASSOS

1. Em `stg_Vendas_Bruto`, **Adicionar Coluna** → **Invocar Função Personalizada**: nome `Dados`, função **`fnLerVendas`**, `conteudo` = coluna `Content`. **OK**.
2. Selecione as colunas `Name` e `Dados` (`Ctrl`+clique) → botão direito → **Remover Outras Colunas**.
3. Renomeie `Name` para `Arquivo_Origem`.
4. Clique no ícone de expandir (duas setas) do cabeçalho `Dados`; desmarque **Usar o nome da coluna original como prefixo**; **OK**.
5. Ajuste o código da etapa de expansão para usar `ColunasAlvo & {"Linha_Origem"}` (assim a lista de colunas não fica "congelada" no código) e acrescente a ordenação. Compare com o código completo abaixo.
6. Feche com **Página Inicial** → **Fechar e Carregar** → **Fechar e Carregar Para...** → **Apenas Criar Conexão**. Mova a consulta para o grupo `04_Staging`.

CÓDIGO M EXATO (CONSULTA STG_VENDAS_BRUTO COMPLETA)

```
let
    Origem = Folder.Files(pPastaVendas),
    SoPlanilhas = Table.SelectRows(Origem, each Text.Lower([Extension]) = ".xlsx"
                                        and not Text.StartsWith([Name], "~$")),
```

```
    // aplica a função de leitura e padronização a cada arquivo
    ComDados = Table.AddColumn(SoPlanilhas, "Dados", each fnLerVendas([Content])),
    SoNomeEDados = Table.SelectColumns(ComDados, {"Name", "Dados"}),
    Renomeada = Table.RenameColumns(SoNomeEDados, {{"Name", "Arquivo_Origem"}}),
    // expandir = anexar: as tabelas de todas as filiais viram uma só
    Anexada = Table.ExpandTableColumn(Renomeada, "Dados", ColunasAlvo & {"Linha_Origem"}),
    Ordenada = Table.Sort(Anexada, {{"Arquivo_Origem", Order.Ascending}, {"Linha_Origem", Order.
    Ascending}})
in
    Ordenada
```

Expandir é anexar. Quando as tabelas são consultas separadas (por exemplo, uma filial manda CSV e as outras `.xlsx`), o mesmo resultado é obtido com **Página Inicial** → **Acrescentar Consultas** → **Acrescentar Consultas como Novas**, que gera `Table.Combine({Consulta1, Consulta2})`. Em ambos os casos o anexo casa as colunas **pelo nome**, não pela posição: por isso a padronização vem antes. O Apêndice C mostra a leitura de CSV.

RESULTADO ESPERADO

O que você deve ver na tela

`stg_Vendas_Bruto` com 13 colunas: `Arquivo_Origem`, as 11 colunas-alvo e `Linha_Origem`. Barra de status do Editor (canto inferior esquerdo): **13 COLUNAS, 27 LINHAS** no simulado (12 da F01, incluindo uma linha totalmente vazia, e 15 da F02). No Excel, o painel **Consultas e Conexões** mostra *Somente conexão*.

Como conferir

- Clique no filtro de `Arquivo_Origem`: 2 valores. A F01 tem `Linha_Origem` de 2 a 13; a F02, de 2 a 16.
- Abra o filtro de `Quantidade`: aparecem valores das **duas** filiais (na F01 a coluna se chamava `Quantidade`, na F02 `Qtd`). Se uma filial vier toda nula, o de-para falhou.
- Na base oficial, compare o total de linhas com a soma das contagens feitas no Excel (`=CONT.VALORES(A:A)-1` de cada arquivo, como na Etapa 1).

ERROS COMUNS E CORREÇÕES

Erro	Correção
Colunas `Dados.ID_Cupom`, `Dados.Data`...	Faltou desmarcar o prefixo. Renomeie no código: a expansão deve usar `ColunasAlvo & {"Linha_Origem"}`.
`Expression.Error: o valor não é do tipo Binary`	No **Invocar Função Personalizada**, escolheu a coluna errada. O argumento é `Content`.
Filial nova ficou de fora	O arquivo não é `.xlsx` (é `.xls` ou `.csv`). Converta, ou siga o Apêndice C.
Total de linhas maior que o esperado	Há linhas vazias formatadas no fim de alguma planilha. Elas serão tratadas na Atividade 3 e aparecerão em `Rejeitados` como `Linha em branco`.

3.4 Conferência das colunas por arquivo

OBJETIVO

Detectar automaticamente, a cada atualização, um cabeçalho que não foi mapeado (por exemplo, uma filial

que passou a mandar `Qtde`).

PASSOS

Crie a consulta nula **`Conf_Colunas`**, coloque no grupo `06_Conferencia` e deixe como *Apenas Criar Conexão* por enquanto (ela será carregada na Atividade 9).

CÓDIGO M EXATO

```
let
    Origem = Folder.Files(pPastaVendas),
    SoPlanilhas = Table.SelectRows(Origem, each Text.Lower([Extension]) = ".xlsx"
                                        and not Text.StartsWith([Name], "~$")),
    Pares = List.Zip({DeParaColunas[Nome_Origem], DeParaColunas[Nome_Alvo]}),
    // nomes que sobram depois de limpar e renomear = colunas que seriam descartadas
    NaoMapeadas = Table.AddColumn(SoPlanilhas, "Colunas_Nao_Mapeadas", each
        let
            Cab = Table.ColumnNames(Table.RenameColumns(
                    Table.TransformColumnNames(fnLerPlanilha([Content]), Text.Trim),
                    Pares, MissingField.Ignore))
        in
            Text.Combine(List.Difference(Cab, ColunasAlvo), ", "), type text),
    Resultado = Table.SelectColumns(NaoMapeadas, {"Name", "Colunas_Nao_Mapeadas"})
in
    Resultado
```

RESULTADO ESPERADO

Uma linha por arquivo, com `Colunas_Nao_Mapeadas` **vazia**. Qualquer texto nessa coluna é uma coluna da filial que seria descartada: acrescente-a ao **`DeParaColunas`** (se tiver nome-alvo) ou registre a decisão de descartá-la no `README.md`.

Como conferir

Teste: em **`DeParaColunas`**, apague temporariamente a linha `{"Loja", "Filial"}`. **`Conf_Colunas`** passa a mostrar `Loja` na F01 e, em **`stg_Vendas_Bruto`**, a coluna `Filial` da F01 fica nula. Desfaça (`Ctrl`+`Z` no Editor Avançado ou digite a linha de novo).

4. Atividade 3 – Limpeza de dados, nulos e duplicidades

Como a consulta fVendas será construída. A partir daqui todas as etapas (Atividades 3 a 7) ficam numa só consulta, **`fVendas`**, que **referencia** **`stg_Vendas_Bruto`**. Cada atividade acrescenta etapas **no fim** da consulta, na ordem do guia. O código completo está no Apêndice A. Ao final de cada atividade, confira a contagem de linhas na barra de status com a tabela da Seção 8.1.

4.1 Criar a consulta fVendas e remover linhas em branco

OBJETIVO

Descartar as linhas totalmente vazias que vêm de planilhas mal formatadas, sem confundi-las com linhas que só têm algumas células vazias.

O QUE PRECISO TER ANTES DE COMEÇAR

`stg_Vendas_Bruto` pronta.

PASSOS

1. No painel **Consultas**, clique com o botão direito em `stg_Vendas_Bruto` → **Referência**. Renomeie a nova consulta para `fVendas` e mova para `05_Saida`.
2. Renomeie a etapa `Fonte` para `Origem`.
3. Clique em `fx` e digite a etapa `SemLinhasEmBranco` do código abaixo.

Atenção

Não use o botão Remover Linhas → Remover Linhas em Branco aqui. Ele só remove a linha se **todas** as colunas estiverem vazias, e as colunas `Arquivo_Origem` e `Linha_Origem` estão sempre preenchidas: nenhuma linha seria removida. A etapa abaixo é o código que o botão gera, com uma diferença: olha apenas as colunas de dados (`Record.SelectFields(_, ColunasAlvo)`).

CÓDIGO M EXATO

```
    Origem = stg_Vendas_Bruto,
    SemLinhasEmBranco = Table.SelectRows(Origem, each not List.IsEmpty(
        List.RemoveMatchingItems(Record.FieldValues(Record.SelectFields(_, ColunasAlvo)), {"", null
    }))),
```

Parte	O que faz
`Record.SelectFields(_, ColunasAlvo)`	Pega, da linha atual (_), só as 11 colunas de dados.
`Record.FieldValues(...)`	Transforma esses campos em uma lista de valores.
`List.RemoveMatchingItems(..., {"", null})`	Tira da lista os vazios e os textos vazios.
`not List.IsEmpty(...)`	Mantém a linha se sobrou **algum** valor.

RESULTADO ESPERADO

27 → **26** linhas no simulado (sai a linha 6 da F01).

Como conferir

Antes de seguir, clique na etapa `Origem` e filtre `ID_Cupom` por **(nulo)**: aparecem 2 linhas da F01 (6 e 13). Clique em `SemLinhasEmBranco`: com o mesmo olhar, só a linha 13 continua (ela tem dados, só falta o cupom). **Não** deixe o filtro como etapa: exclua-o.

4.2 Tratar nulos nas colunas críticas e não críticas (D5)

OBJETIVO

Aplicar a regra do dicionário de dados: `ID_Cupom` e `Quantidade` são **críticos** (sem eles a venda não pode ser medida) e `Desconto` é **não crítico** ("0 quando não houver").

O QUE PRECISO TER ANTES DE COMEÇAR

Etapa `SemLinhasEmBranco`.

QUAL TRATAMENTO PARA CADA COLUNA

Coluna	Nulo ou vazio	Por quê
`ID_Cupom`	**descartar** → `Rejeitados`	Sem cupom não há ticket médio nem contagem de vendas; é a chave da transação.
`Quantidade`	**descartar** → `Rejeitados` (vazia ou 0)	D5. Não se "chuta" quantidade: a Etapa 1 já registrou a confirmação pendente.
`Desconto`	**substituir por 0**	Regra explícita do dicionário (Seção 4.2).
`Valor_Unit`, `Data`, `Hora`	tratados na Atividade 4	Dependem da conversão de tipo para saber se são válidos.

PASSOS

1. Filtro de `ID_Cupom` → desmarque **(nulo)** → **OK**. Edite a etapa para incluir também o teste de texto vazio (código abaixo). Renomeie para `SemCupomVazio`.
2. Filtro de `Quantidade` → desmarque **(nulo)** e `0` → **OK**. Renomeie para `SemQtdVazia` e confira o código.
3. Selecione a coluna `Desconto` → **Transformar** → **Substituir Valores**: **Valor a Ser Localizado** `null`; **Substituir Por** `0`. Renomeie para `DescontoNuloZero`.

CÓDIGO M EXATO

```
    SemCupomVazio = Table.SelectRows(SemLinhasEmBranco, each [ID_Cupom] <> null
                                    and Text.Trim(Text.From([ID_Cupom])) <> ""),
    SemQtdVazia = Table.SelectRows(SemCupomVazio, each [Quantidade] <> null and [Quantidade] <> 0),
    DescontoNuloZero = Table.ReplaceValue(SemQtdVazia, null, 0, Replacer.ReplaceValue, {"Desconto"}),
```

RESULTADO ESPERADO

■ EXEMPLO SIMULADO – não são dados oficiais do projeto

Etapa	Linhas	Quem saiu
`SemLinhasEmBranco`	26	–
`SemCupomVazio`	25	F01, linha 13 (Croissant sem cupom)
`SemQtdVazia`	22	F01 linha 8 (Cookie, vazia); F02 linha 6 (Café Gelado, vazia); F02 linha 7 (Frappé, 0)
`DescontoNuloZero`	22	ninguém sai; F01 linha 5 (Latte) passa de `null` a `0` em `Desconto`

Como conferir

Exibir → **Qualidade da Coluna**: na etapa `DescontoNuloZero`, as colunas `ID_Cupom`, `Quantidade` e `Desconto` mostram **Vazio 0%**. Na barra de status, mude **Criação de perfil de coluna com base nas 1000 primeiras linhas** para **. . . com base no conjunto de dados inteiro** quando usar a base oficial.

ERROS COMUNS E CORREÇÕES

Erro	Correção
Substituiu nulos de `Quantidade` por 0 ou por 1	Desfaça. Quantidade é crítica: a linha sai e é registrada, não é inventada.
Substituir Valores não acha `null`	Digite `null` em minúsculas, sem aspas. Com aspas o Power Query procura o *texto* "null".
Removeu a linha com **Remover Linhas** → **Remover Erros**	Isso trata erros, não nulos. Use os filtros desta seção.

4.3 Remover duplicidades pela chave da transação (D8)

OBJETIVO

Eliminar a segunda ocorrência de uma mesma venda lançada duas vezes: **mesmo cupom, mesmo produto e mesma hora** (Seção 4.3 da especificação).

O QUE PRECISO TER ANTES DE COMEÇAR

Etapa `DescontoNuloZero`. Entender a diferença entre item repetido e cupom repetido: um cupom com dois produtos diferentes (F02, linhas 2 e 3) **não** é duplicata.

PASSOS

1. Selecione, com `Ctrl`+clique e nesta ordem, as colunas `ID_Cupom`, `Produto` e `Hora`.
2. **Página Inicial** → **Remover Linhas** → **Remover Duplicatas**. Renomeie a etapa para `SemDuplicatasExatas`.

CÓDIGO M EXATO

```
    // 1ª passagem: duplicatas idênticas (ainda com os textos como vieram da filial)
    SemDuplicatasExatas = Table.Distinct(DescontoNuloZero, {"ID_Cupom", "Produto", "Hora"}),
```

Atenção

Esta é só a primeira passagem. `Table.Distinct` compara os textos exatamente: `"Torta Salgada"` e `"Torta Salgada␣"` são diferentes para ele. A linha 12 da F01 escapa aqui e só é pega na **segunda passagem** (Seção 7.3), feita depois da padronização de textos e com a chave oficial da especificação: `{"ID_Cupom", "Cod_Produto", "Hora"}`. Fazer as duas é mais seguro do que confiar em uma só.

RESULTADO ESPERADO

22 → **21** linhas: sai a linha 4 da F01, cópia exata da linha 3 (Pão de Queijo, cupom `F01-000201`, 07:10). Fica a primeira ocorrência.

Como conferir

Na etapa `DescontoNuloZero`, selecione as 3 colunas da chave → **Página Inicial** → **Manter Linhas** → **Manter Duplicatas**: aparecem as linhas 3 e 4 da F01. Exclua essa etapa de teste.

ERROS COMUNS E CORREÇÕES

Erro	Correção
Selecionou só `ID_Cupom` antes de remover duplicatas	Itens diferentes do mesmo cupom sumiram (F02 linha 3). Edite a etapa para as 3 colunas.
Removeu duplicatas com **todas** as colunas selecionadas	`Linha_Origem` é sempre diferente, então nada sai. Use só a chave.

5. Atividade 4 – Ajuste de tipos e localidade pt-BR

Por que tipos importam. Até aqui as colunas têm o tipo `ABC123` (*qualquer*): uma mesma coluna mistura datas verdadeiras e textos como `3-3-2026`. Somas, filtros de período e medidas DAX exigem um tipo único por coluna. A localidade diz **como ler** o texto: em pt-BR, `"8,50"` é oito e cinquenta; em en-US, a vírgula é separador de milhar e o resultado vira **850**.

5.1 Datas e horas em formatos diferentes (D1)

OBJETIVO

Converter `Data` e `Hora` para os tipos `date` e `time`, lendo os três formatos de data do defeito D1 e transformando datas inexistentes em `null` (sem erro).

O QUE PRECISO TER ANTES DE COMEÇAR

Etapa `SemDuplicatasExatas`.

Atenção

Por que não basta Tipo de Dados → Data. A conversão automática lê `03/03/2026` e `2026-03-03`, mas a reação a `3-3-2026` depende da localidade, e `31/02/2026` vira **Erro**. Erros espalhados contaminam as etapas seguintes. A função abaixo trata cada formato de forma explícita, como a fórmula 5 da Etapa 1, e devolve `null` para o que não for uma data real.

PASSOS

1. Crie as funções **`fnData`** e **`fnHora`** (consulta nula + Editor Avançado), no grupo `02_Funcoes`.
2. Em **`fVendas`**, clique em `fx` e digite a etapa `DatasHoras`.
3. Filtre `Data`: desmarque **(nulo)**. Renomeie para `SemDataInvalida`.
4. **Filtros de Data → Entre**: `01/01/2026` e `30/06/2026` (período oficial, Seção 3.3). Renomeie para `NoPeriodo` e confira o código.
5. Filtre `Hora`: desmarque **(nulo)**. Renomeie para `SemHoraInvalida`.

CÓDIGO M EXATO

```
// fnData -- aceita data, data/hora, número de série do Excel ou texto d-m-aaaa / aaaa-m-d
(valor as any) as nullable date =>
let
    Resultado =
        if valor = null then null
        else if valor is date then valor
        else if valor is datetime then Date.From(valor)
        else if valor is number then Date.From(valor)          // série do Excel (46084)
        else
            let
                Texto = Text.Replace(Text.Trim(Text.From(valor)), "/", "-"),
                Partes = Text.Split(Texto, "-"),
                P1 = Number.From(Partes{0}),
                P2 = Number.From(Partes{1}),
                P3 = Number.From(Partes{2}),
                // se a 1ª parte tem 4 dígitos, é ano-mês-dia; senão, dia-mês-ano
                Data = if Text.Length(Partes{0}) = 4 then #date(P1, P2, P3) else #date(P3, P2, P1)
            in
                Data
in
    // #date(2026, 2, 31) gera erro: try...otherwise transforma em null
    try Resultado otherwise null

// fnHora -- aceita hora, data/hora, fração do dia ou texto "hh:mm"
(valor as any) as nullable time =>
let
    Resultado =
        if valor = null then null
        else if valor is time then valor
```

```
        else if valor is datetime then DateTime.Time(valor)
        else if valor is number then Time.From(valor - Number.RoundDown(valor))
        else Time.FromText(Text.Trim(Text.From(valor)), "pt-BR")
in
    try Resultado otherwise null

// etapas em fVendas
    DatasHoras = Table.TransformColumns(SemDuplicatasExatas,
                  {{"Data", fnData, type nullable date}, {"Hora", fnHora, type nullable time}}),
    SemDataInvalida = Table.SelectRows(DatasHoras, each [Data] <> null),
    NoPeriodo = Table.SelectRows(SemDataInvalida, each [Data] >= #date(2026, 1, 1)
                                                  and [Data] <= #date(2026, 6, 30)),
    SemHoraInvalida = Table.SelectRows(NoPeriodo, each [Hora] <> null),
```

Parte	Equivalente na Etapa 1
`valor is date` / `is datetime`	`SE(ÉNÚM(v); v; ...)`: se já é data, usa como está.
`Text.Replace(..., "/", "-")`	`SUBSTITUIR(...;"/";"-")`: um só separador para os três formatos.
`Text.Split(Texto, "-")`	`LOCALIZAR` + `ESQUERDA/EXT`: separa as três partes de uma vez (`Partes{0}` é a primeira, pois a contagem começa em zero).
`Text.Length(Partes{0}) = 4`	`SE(a > 31; ...)`: decide se o texto começa pelo ano.
`try ... otherwise null`	`SEERRO(...;"Data inválida")` mais o teste `ok`: no M, `#date` **recusa** 31/02 em vez de "rolar" para março, então basta capturar o erro.

RESULTADO ESPERADO

■ EXEMPLO SIMULADO – não são dados oficiais do projeto

Arquivo	Linha	Original	Depois	Destino
F02	4	3-3-2026 (texto)	03/03/2026	segue
F02	5	2026-03-03 (texto)	03/03/2026	segue (sai depois, na Atividade 6)
F01	5	15/01/2026 (texto)	15/01/2026	segue
F02	13	31/02/2026 (texto)	`null`	sai em `SemDataInvalida`
F01	9	10/07/2026 (data)	10/07/2026	sai em `NoPeriodo` (julho, fora do semestre)
F02	4	09:10 (texto)	09:10	hora segue

Contagem: `DatasHoras` 21; `SemDataInvalida` **20**; `NoPeriodo` **19**; `SemHoraInvalida` 19.

Como conferir

O ícone do cabeçalho de `Data` deve ser um **calendário** e o de `Hora` um **relógio**. Na etapa `DatasHoras`, **Exibir** → **Qualidade da Coluna** mostra `Data` com **Erro 0%** e **Vazio** > 0 (o 31/02). Se aparecer erro, a etapa não está usando `fnData`.

ERROS COMUNS E CORREÇÕES

Erro	Correção
Datas com dia e mês trocados (05/03 virou 03/05)	Você usou **Tipo** → **Data** com localidade en-US. Use `fnData`, que não depende da localidade.
Hora 07:45 virou 00:00	A hora veio como data/hora e foi convertida com `Date.From`. Use `fnHora` (`DateTime.Time`).
`Expression.Error: fnData não reconhecido`	O nome da função tem maiúscula diferente (`FnData`). M diferencia maiúsculas.
Ano com dois dígitos (3-3-26)	Vira o ano 26 d.C. e sai como *fora do período*. Está correto: aparece em `Rejeitados` para ser esclarecido.

5.2 Números com vírgula guardados como texto (D6)

OBJETIVO

Converter `Quantidade` em inteiro e `Valor_Unit` e `Desconto` em decimal, lendo `"8,50"` e `"10,50"` como números, e separar o que não for número.

O QUE PRECISO TER ANTES DE COMEÇAR

Etapa `SemHoraInvalida`.

PASSOS

1. Selecione `ID_Cupom` → clique no ícone de tipo do cabeçalho → **Texto**.
2. Selecione `Quantidade` → clique com o botão direito → **Alterar Tipo** → **Usando a Localidade...** → **Tipo de Dados: Número Inteiro**; **Localidade: Português (Brasil)**.
3. Repita para `Valor_Unit` e `Desconto` com **Número Decimal**. Junte as três etapas de tipo em uma só (código abaixo) e chame-a `TiposPtBR`.
4. Selecione `Quantidade`, `Valor_Unit` e `Desconto` → **Página Inicial** → **Remover Linhas** → **Remover Erros**. Renomeie para `SemErrosDeTipo`.
5. Digite a etapa `QtdEValorValidos`.

CÓDIGO M EXATO

```
    TiposPtBR = Table.TransformColumnTypes(SemHoraInvalida,
                  {{"ID_Cupom", type text}, {"Quantidade", Int64.Type},
                   {"Valor_Unit", type number}, {"Desconto", type number}}, "pt-BR"),
    // texto que não é número (ex.: "n/d") vira Erro na conversão: a linha sai aqui
    SemErrosDeTipo = Table.RemoveRowsWithErrors(TiposPtBR, {"Quantidade", "Valor_Unit", "Desconto"}),

    QtdEValorValidos = Table.SelectRows(SemErrosDeTipo, each [Quantidade] > 0 and [Valor_Unit] <>
    null),
```

RESULTADO ESPERADO

■ EXEMPLO SIMULADO – não são dados oficiais do projeto

Arquivo	Linha	Valor_Unit original	Depois	Observação
F02	15	`"8,50"` (texto)	8,5	D6 resolvido (na Etapa 1 só foi registrado)
F01	5	`"10,50"` (texto)	10,5	D6 resolvido
F01	10	`"n/d"` (texto)	**Error**	sai em `SemErrosDeTipo`
F02	8	65 (número)	65	segue: preço fora da referência (D9) não é motivo de descarte

Contagem: `TiposPtBR` 19; `SemErrosDeTipo` **18**; `QtdEValorValidos` 18.

Como conferir

- Ícones: `ID_Cupom` ABC, `Quantidade` 123, `Valor_Unit` e `Desconto` `1.2`.
- Clique no cabeçalho de `Valor_Unit` → **Exibir** → **Perfil da Coluna**: mínimo 5, máximo 65 no simulado. Se o máximo for 850 ou 1050, a localidade está errada.
- Antes de `SemErrosDeTipo`, **Manter Linhas** → **Manter Erros** mostra só a linha 10 da F01. Exclua a etapa de teste.

ERROS COMUNS E CORREÇÕES

Erro	Correção
`8,50` virou `850`	Mudou o tipo pelo ícone, sem localidade. Edite a etapa e acrescente `, "pt-BR"` antes do último parêntese.
Usou **Substituir Valores** para trocar `,` por `.`	Funciona no seu computador e quebra no do colega com outra configuração. Use a localidade.
`Remover Erros` sem selecionar colunas	Remove linhas com erro em **qualquer** coluna, inclusive as que você não pretendia. Selecione as três.
`Int64.Type` arredondou `1,5` para `2`	Quantidade fracionária é dado inválido: registre na auditoria. Não é esperado na base do projeto.

6. Atividade 5 – Padronização de textos e maiúsculas

OBJETIVO

Remover espaços das pontas (D3), unificar maiúsculas e minúsculas de categoria e produto (D2) e trazer pagamento e canal para a grafia oficial das listas da Etapa 1.

O QUE PRECISO TER ANTES DE COMEÇAR

Etapa `QtdEValorValidos`; consultas **`lst_Pagamento`** e **`lst_Canal`** (Seção 2.3).

QUAL TRATAMENTO PARA CADA CAMPO

Campo	Tratamento no Power Query	Equivalente na Etapa 1
`ID_Cupom`	**nenhum**	idem: identificador não se altera
`Filial`	`Text.Trim`	`ARRUMAR` dentro do `PROCX`
`Produto`, `Categoria`	`Text.Trim` + `Text.Proper`	`PRI.MAIUSCULA(ARRUMAR())`
`Pagamento`, `Canal`	`Text.Trim` + busca na lista oficial (função **`fnValorOficial`**)	`PROCX` na lista com `"Fora da lista"`

PASSOS

1. Selecione `Filial`, `Produto`, `Categoria`, `Pagamento` e `Canal` → **Transformar** → **Formatar** → **Aparar**. Renomeie para `TextosAparados`.
2. Selecione `Produto` e `Categoria` → **Transformar** → **Formatar** → **Colocar Cada Palavra em Maiúscula**. Renomeie para `PrimeiraMaiuscula`.

3. Crie a função **fnValorOficial** (grupo 02_Funcoes).
4. Digite as etapas PagamentoOficial e CanalOficial.

CÓDIGO M EXATO

```
// fnValorOficial -- devolve a grafia da lista oficial, ignorando maiúsculas/minúsculas
(valor as nullable text, lista as list) as nullable text =>
let
    Encontrados = List.Select(lista, each Text.Upper(_) = Text.Upper(valor))
in
    if valor = null or List.IsEmpty(Encontrados) then valor else Encontrados{0}

// etapas em fVendas
    TextosAparados = Table.TransformColumns(QtdEValorValidos,
        {{"Filial", Text.Trim, type text}, {"Produto", Text.Trim, type text},
         {"Categoria", Text.Trim, type text}, {"Pagamento", Text.Trim, type text},
         {"Canal", Text.Trim, type text}}),
    PrimeiraMaiuscula = Table.TransformColumns(TextosAparados,
        {{"Produto", Text.Proper, type text}, {"Categoria", Text.Proper, type text}}),
    PagamentoOficial = Table.TransformColumns(PrimeiraMaiuscula,
        {{"Pagamento", each fnValorOficial(_, lst_Pagamento), type text}}),
    CanalOficial = Table.TransformColumns(PagamentoOficial,
        {{"Canal", each fnValorOficial(_, lst_Canal), type text}}),
```

Atenção

Text.Trim não é igual a ARRUMAR. Text.Trim remove espaços **só das pontas**; ARRUMAR também reduz espaços repetidos *no meio* ("Café␣␣Gelado"). Se a base oficial tiver espaços duplos internos, use a função **fnLimparTexto** do Apêndice B. Pelo mesmo motivo de cuidado, Text.Proper não serve para pagamento: transformaria Vale-refeição em Vale-Refeição; por isso a busca na lista oficial.

RESULTADO ESPERADO

■ EXEMPLO SIMULADO – não são dados oficiais do projeto

Arquivo	Linha	Antes	Depois	Observação
F01	5	SP-Centro␣	SP-Centro	D3 (filial)
F01	5	latte␣ / quente	Latte / Quente	D3 e D2
F01	5	débito / balcão	Débito / Balcão	grafia oficial da lista
F02	4	Cappuccino␣ / quente	Cappuccino / Quente	D3 e D2
F02	15	Bolo do Dia (fatia)	Bolo Do Dia (Fatia)	só aparência; a busca do código usa chave em maiúsculas
F02	14	App	App	fora da lista: **mantido** e pendente na auditoria da Etapa 1

Nenhuma linha sai nesta atividade: continuam **18**.

Como conferir

Abra o filtro de Categoria: só Frio, Padaria e Quente. Filtro de Pagamento: só valores da lista oficial (no simulado, Crédito, Débito, Pix, Vale-refeição). Filtro de Canal: App, Balcão, Delivery – App é o único fora da lista e já está na auditoria da Etapa 1.

ERROS COMUNS E CORREÇÕES

Erro	Correção
`Expression.Error: não foi possível converter o valor 123 para o tipo Text`	Há número numa coluna de texto. Troque `Text.Trim` por `each Text.Trim(Text.From(_))`.
Aplicou `Text.Proper` em `ID_Cupom`	Desfaça: `F01-000201` não muda, mas um código como `f01-aB` mudaria. Identificador não se trata.
`Débito` continua `débito`	**`lst_Pagamento`** é uma tabela, não uma lista. Confira se o código termina em `[Pagamento]`.
Vontade de trocar `App` por `Delivery`	Só depois de confirmado pelo professor; aí acrescente a regra e anote a decisão no `README.md`.

7. Atividade 6 – Mesclagem com o De-Para de Filiais

7.1 Obter o código oficial da filial (D4)

OBJETIVO

Trocar as variações do nome da filial (`SP-ZSul`, `Zona Sul`, `SP Zona Sul...`) pelo código oficial `F01`–`F05`, cruzando com **`cad_DeParaFilial`**.

O QUE PRECISO TER ANTES DE COMEÇAR

Etapa `CanalOficial`; consulta **`cad_DeParaFilial`** com a coluna `Chave_Filial`.

PASSOS

1. **Adicionar Coluna** → **Coluna Personalizada**: nome `Chave_Filial`, fórmula `Text.Upper([Filial])`. Renomeie a etapa para `ChaveFilial`.
2. **Página Inicial** → **Mesclar Consultas** (não *como Nova*). Tabela de cima: **`fVendas`**, clique na coluna `Chave_Filial`. Tabela de baixo: **`cad_DeParaFilial`**, clique em `Chave_Filial`. **Tipo de Junção: Externa esquerda (todas da primeira, correspondentes da segunda)**. **OK**.
3. Leia a mensagem na parte de baixo da janela antes do **OK**: *A seleção corresponde a X de Y linhas da primeira tabela*. No simulado: **17 de 18**.
4. Renomeie a coluna nova para `DePara` e a etapa para `MesclaFilial`. Expanda `DePara` marcando só `Cod_Filial`, sem prefixo. Renomeie para `ExpandeFilial`.
5. Filtre `Cod_Filial`: desmarque **(nulo)**. Renomeie para `ComFilial`.

CÓDIGO M EXATO

```
    ChaveFilial = Table.AddColumn(CanalOficial, "Chave_Filial", each Text.Upper([Filial]), type text
    ),
    MesclaFilial = Table.NestedJoin(ChaveFilial, {"Chave_Filial"}, cad_DeParaFilial, {"Chave_Filial"
    },
                                    "DePara", JoinKind.LeftOuter),
    ExpandeFilial = Table.ExpandTableColumn(MesclaFilial, "DePara", {"Cod_Filial"}, {"Cod_Filial"}),
    ComFilial = Table.SelectRows(ExpandeFilial, each [Cod_Filial] <> null),
```

Tipo de junção	Quando usar neste projeto
Externa esquerda (`LeftOuter`)	Mantém todas as vendas; a que não encontra par recebe `null` e pode ser filtrada e registrada. É a escolha aqui.
Interna (`Inner`)	Descarta em silêncio a venda sem par: o `Z. Sul` sumiria sem registro. Evite.
Anti esquerda (`LeftAnti`)	Devolve **só** as linhas sem par. Usada na Atividade 8 para montar `Rejeitados`.

RESULTADO ESPERADO

■ EXEMPLO SIMULADO – não são dados oficiais do projeto

Filial (depois do Trim)	Chave_Filial	Cod_Filial
SP-Centro	SP-CENTRO	F01
SP-Zona Sul / SP-ZSul / Zona Sul	SP-ZONA SUL / SP-ZSUL / ZONA SUL	F02
Z. Sul (F02, linha 12)	Z. SUL	**null** → sai em `ComFilial`

Contagem: `ExpandeFilial` 18; `ComFilial` **17**. (A grafia `SP Zona Sul` estava na linha 6 da F02, que já saiu por quantidade vazia.)

Como conferir

- `ExpandeFilial` tem **o mesmo número de linhas** que `CanalOficial` (18). Se aumentou, o de-para tem chave repetida e a mesclagem multiplicou vendas.
- Coerência com o arquivo: filtre `Arquivo_Origem` pela F02 e abra o filtro de `Cod_Filial`: só deve aparecer `F02`. Repita para a F01 (só `F01`). Um código diferente do que está no nome do arquivo indica erro no de-para. Remova os filtros de teste.

Atenção

O de-para continua sendo da Etapa 1. Se o professor confirmar que `Z. Sul` é a Zona Sul, acrescente a linha `Z. Sul | F02` na `Tab_DePara_Filial` do arquivo `CafeData_01_Excel.xlsx`, salve, e clique em **Atualizar Tudo** no arquivo da Etapa 2: a venda volta para **`fVendas`** sozinha. Não edite o de-para dentro do Power Query: haveria duas versões da verdade.

ERROS COMUNS E CORREÇÕES

Erro	Correção
A seleção corresponde a 0 linhas	Mesclou `Filial` com `Nome_Origem` (caixa diferente). Use as duas colunas `Chave_Filial`.
Número de linhas aumentou após expandir	Chave repetida em **`cad_DeParaFilial`**. Corrija a `Tab_DePara_Filial` na Etapa 1.
Escolheu **Mesclar Consultas como Novas**	Criou a consulta `Mesclar1`. Exclua-a e refaça com **Mesclar Consultas** dentro de **`fVendas`**.
Usou **Correspondência difusa**	Ela pode ligar `Z. Sul` a `Zona Sul` "por semelhança", sem confirmação. Não use neste projeto.

7.2 Obter o código do produto no cadastro (D7)

OBJETIVO

Trocar o nome do produto pelo código `P01`–`P18`, que é a chave da dimensão **`dProduto`**. Produto sem cadastro sai e é registrado.

O QUE PRECISO TER ANTES DE COMEÇAR

Etapa `ComFilial`; consulta **`cad_ProdutoChave`**.

PASSOS

1. Coluna personalizada `Chave_Produto` com `Text.Upper([Produto])`. Etapa `ChaveProduto`.
2. **Mesclar Consultas** com **`cad_ProdutoChave`** por `Chave_Produto`, externa esquerda (no simulado: **16 de 17**). Nome da coluna `Cadastro`; etapa `MesclaProduto`.
3. Expanda só `Cod_Produto`, sem prefixo (`ExpandeProduto`) e filtre os nulos (`ComProduto`).

CÓDIGO M EXATO

```
    ChaveProduto = Table.AddColumn(ComFilial, "Chave_Produto", each Text.Upper([Produto]), type text
    ),
    MesclaProduto = Table.NestedJoin(ChaveProduto, {"Chave_Produto"}, cad_ProdutoChave, {"
    Chave_Produto"},
                                     "Cadastro", JoinKind.LeftOuter),
    ExpandeProduto = Table.ExpandTableColumn(MesclaProduto, "Cadastro", {"Cod_Produto"}, {"
    Cod_Produto"}),
    ComProduto = Table.SelectRows(ExpandeProduto, each [Cod_Produto] <> null),
```

RESULTADO ESPERADO

17 → **16** linhas: sai a linha 5 da F02 (`Capuccino`, grafia errada). `Bolo Do Dia (Fatia)` encontra `P16`, porque as duas chaves ficam `BOLO DO DIA (FATIA)`.

Como conferir

Abra o filtro de `Cod_Produto`: só códigos `P01`–`P18`, nenhum (`nulo`). Compare com a Etapa 1: lá, `Capuccino` virou `Não cadastrado`; aqui, a linha sai e aparecerá em **`Rejeitados`** com o motivo `Produto não cadastrado`.

7.3 Segunda passagem de duplicidades pela chave oficial (D8)

OBJETIVO

Aplicar a chave da especificação, `{"ID_Cupom", "Cod_Produto", "Hora"}`, agora que os produtos estão padronizados, pegando duplicatas que diferiam só em espaços ou maiúsculas.

PASSOS

1. Digite as etapas `OrdemOriginal` e `SemDuplicatas` do código abaixo.

CÓDIGO M EXATO

```
    // garante que a 1ª ocorrência mantida seja a de menor Linha_Origem
    OrdemOriginal = Table.Buffer(Table.Sort(ComProduto,
                      {{"Arquivo_Origem", Order.Ascending}, {"Linha_Origem", Order.Ascending}})),
    SemDuplicatas = Table.Distinct(OrdemOriginal, {"ID_Cupom", "Cod_Produto", "Hora"}),
```

`Table.Distinct` mantém a primeira linha de cada chave, mas mesclagens podem mudar a ordem das linhas. `Table.Sort` + `Table.Buffer` fixam a ordem antes de remover.

RESULTADO ESPERADO

16 → **15** linhas: sai a linha 12 da F01 (`"Torta Salgada␣"` / `padaria`), que depois do Trim e do Proper é igual à linha 11.

Como conferir

Na etapa `OrdemOriginal`, filtre `ID_Cupom` = `F01-000207`: 2 linhas (11 e 12). Em `SemDuplicatas`, só a linha 11. Remova o filtro de teste.

8. Atividade 7 – Criação de colunas personalizadas

OBJETIVO

Calcular `Valor_Bruto` = `Quantidade` × `Valor_Unit` e `Valor_Liquido` = `Valor_Bruto` − `Desconto` (Seção 4.2 da especificação) e deixar **`fVendas`** com as colunas na ordem final.

O QUE PRECISO TER ANTES DE COMEÇAR

Etapa `SemDuplicatas`, com `Quantidade`, `Valor_Unit` e `Desconto` já numéricos (Atividade 4).

PASSOS

1. **Adicionar Coluna** → **Coluna Personalizada**. **Nome da nova coluna**: `Valor_Bruto`. Fórmula (dê duplo clique nos nomes em **Colunas disponíveis** para inseri-los): `Number.Round([Quantidade] * [Valor_Unit], 2)`. **OK**.
2. Na barra de fórmulas, acrescente `, type number` antes do último parêntese (a coluna personalizada nasce com tipo *qualquer*). Renomeie a etapa para `ValorBruto`.
3. Repita para `Valor_Liquido` com a fórmula `Number.Round([Valor_Bruto] - [Desconto], 2)`. Etapa `ValorLiquido`.
4. **Página Inicial** → **Escolher Colunas**: marque só as 14 colunas da lista abaixo. Depois arraste os cabeçalhos para a ordem do código, ou edite o código. Etapa `FormatoFinal`.

CÓDIGO M EXATO

```
    ValorBruto = Table.AddColumn(SemDuplicatas, "Valor_Bruto",
                    each Number.Round([Quantidade] * [Valor_Unit], 2), type number),
    ValorLiquido = Table.AddColumn(ValorBruto, "Valor_Liquido",
                    each Number.Round([Valor_Bruto] - [Desconto], 2), type number),
    FormatoFinal = Table.SelectColumns(ValorLiquido,
        {"ID_Cupom", "Data", "Hora", "Cod_Filial", "Cod_Produto", "Quantidade", "Valor_Unit",
         "Desconto", "Valor_Bruto", "Valor_Liquido", "Pagamento", "Canal",
         "Arquivo_Origem", "Linha_Origem"})
in
    FormatoFinal
```

Coluna que sai	Por quê
`Filial`, `Chave_Filial`	Substituídas por `Cod_Filial`; o nome oficial está em **`dFilial`**.
`Produto`, `Chave_Produto`	Substituídas por `Cod_Produto`; nome, preço e custo estão em **`dProduto`**.
`Categoria`	A categoria oficial vem de **`dProduto`** (lição da Etapa 1: `Chá Gelado` digitado como `Quente` é `Frio` no cadastro). Ela foi padronizada na Atividade 5 para conferência, mas não deve competir com o cadastro no modelo da Etapa 3.

`Arquivo_Origem` e `Linha_Origem` ficam: são a rastreabilidade. `Custo_Total` e margem **não** são criados aqui: dependem do cadastro e serão medidas DAX na Etapa 3 (`SUMX` com `RELATED`).

RESULTADO ESPERADO

■ EXEMPLO SIMULADO – não são dados oficiais do projeto

Arquivo	Linha	Produto	Qtd	Valor_ Unit	Desconto	Bruto	Líquido
F01	7	P05 Mocha	1	12,00	1,00	12,00	11,00
F02	2	P01 Café Expresso	2	6,50	0,00	13,00	13,00
F02	14	P06 Choc. Quente	2	11,00	1,00	22,00	21,00
F02	15	P16 Bolo do Dia	1	8,50	0,00	8,50	8,50

`fVendas`: **15 linhas, 14 colunas**. Totais: Quantidade **19**; Valor_Bruto **212,00**; Desconto **2,00**; Valor_Liquido **210,00**. Gabarito completo no Apêndice D.

Como conferir

- Clique no cabeçalho de `Valor_Bruto` → **Exibir** → **Perfil da Coluna**: a estatística *Soma* deve ser 212 no simulado.
- Prova real: `Valor_Bruto` − `Valor_Liquido` somados por toda a tabela = soma de `Desconto` (2,00).
- Nenhum `Valor_Liquido` negativo: **Filtros de Número** → **Menor Que** 0 devolve 0 linhas (remova o filtro de teste).

ERROS COMUNS E CORREÇÕES

Erro	Correção
`Valor_Bruto` com ícone `ABC123`	Faltou `, type number`. A Etapa 3 trataria a coluna como texto.
`Expression.Error: não é possível aplicar o operador * a Text e Number`	`Valor_Unit` ainda é texto: a etapa `TiposPtBR` não está antes desta.
Soma com 0,0000001 sobrando	Faltou `Number.Round(..., 2)`. Decimais binários acumulam resíduos.
`Escolher Colunas` gerou `Table.RemoveColumns`	Funciona, mas uma coluna nova na origem passaria direto. Prefira `Table.SelectColumns`, como no código.

8.1 Contagem de linhas etapa por etapa

Use esta tabela para localizar onde uma linha foi descartada. Clique em cada etapa e leia a barra de status do Editor.

EXEMPLO SIMULADO – não são dados oficiais do projeto

Etapa de fVendas	Linhas	Saíram	Quais (arquivo/linha)
`Origem`	27	–	F01: 12; F02: 15
`SemLinhasEmBranco`	26	1	F01/6
`SemCupomVazio`	25	1	F01/13
`SemQtdVazia`	22	3	F01/8; F02/6; F02/7
`SemDuplicatasExatas`	21	1	F01/4
`SemDataInvalida`	20	1	F02/13
`NoPeriodo`	19	1	F01/9
`SemErrosDeTipo`	18	1	F01/10
`ComFilial`	17	1	F02/12
`ComProduto`	16	1	F02/5
`SemDuplicatas`	15	1	F01/12
`FormatoFinal`	**15**	**12**	total descartado = linhas de **`Rejeitados`**

9. Atividade 8 – Tratamento de exceções e consulta de rejeitados

9.1 Criar a consulta Rejeitados

OBJETIVO

Registrar, com o motivo, cada linha lida que não chegou a **`fVendas`**. É o equivalente automático da `Tab_Auditoria` da Etapa 1.

O QUE PRECISO TER ANTES DE COMEÇAR

`fVendas` pronta, mantendo `Arquivo_Origem` e `Linha_Origem`.

IDEIA

1. **Quais linhas?** As de **`stg_Vendas_Bruto`** que não têm par em **`fVendas`** pela chave `Arquivo_Origem` + `Linha_Origem`: uma junção **anti esquerda**. Isso garante, por construção, que lidas = aceitas + rejeitadas.
2. **Por quê?** Uma função `Diagnostico` refaz, sobre os valores **originais**, os mesmos testes de **`fVendas`**, **na mesma ordem**; o primeiro teste que falha é o motivo. Se todos passam, a linha só pode ter saído por duplicidade.

PASSOS

1. Botão direito em **`stg_Vendas_Bruto`** → **Referência**. Renomeie para `Rejeitados`; grupo `05_Saida`.
2. **Página Inicial** → **Mesclar Consultas**: em cima, selecione `Arquivo_Origem` e depois, com `Ctrl`, `Linha_Origem`; embaixo, **`fVendas`**, as mesmas duas colunas **na mesma ordem**. **Tipo de Junção: Anti esquerda (linhas somente na primeira)**. Antes do **OK**, a janela informa quantas linhas têm par (no simulado, 15 de 27): as 12 que **não** têm par são as rejeitadas. **OK**.
3. Remova a coluna da mesclagem (ela só tem tabelas vazias).
4. Abra o **Editor Avançado** e complete com o código abaixo (as listas, a função `Diagnostico` e as etapas `ComMotivo` e `Ordenada`).

CÓDIGO M EXATO (CONSULTA REJEITADOS COMPLETA)

```
let
    Origem = stg_Vendas_Bruto,
    // 1. linhas lidas que NÃO estão em fVendas (junção anti esquerda)
    Mescla = Table.NestedJoin(Origem, {"Arquivo_Origem", "Linha_Origem"},
                              fVendas, {"Arquivo_Origem", "Linha_Origem"}, "Aceita", JoinKind.
    LeftAnti),
    SoRejeitadas = Table.RemoveColumns(Mescla, {"Aceita"}),
    // 2. apoio ao diagnóstico (List.Buffer: lê a lista uma vez só)
    Filiais = List.Buffer(cad_DeParaFilial[Chave_Filial]),
    Produtos = List.Buffer(cad_ProdutoChave[Chave_Produto]),
    ENumero = (v) => v = null or (try (Number.From(v, "pt-BR") is number) otherwise false),
    Chave = (v) => if v = null then null else Text.Upper(Text.Trim(Text.From(v))),
    // 3. mesma ordem de regras de fVendas: o primeiro teste que falha é o motivo
    Diagnostico = (r as record) as text =>
        let
            Valores = List.RemoveMatchingItems(Record.FieldValues(Record.SelectFields(r, ColunasAlvo
    )), {"", null}),
            Data = fnData(r[Data])
        in
            if List.IsEmpty(Valores) then "Linha em branco"
            else if r[ID_Cupom] = null or Text.Trim(Text.From(r[ID_Cupom])) = "" then "ID_Cupom
    vazio"
            else if r[Quantidade] = null then "Quantidade vazia"
            else if r[Quantidade] = 0 then "Quantidade zero"
            else if Data = null then "Data inválida"
            else if Data < #date(2026, 1, 1) or Data > #date(2026, 6, 30) then "Data fora do período
    "
            else if fnHora(r[Hora]) = null then "Hora inválida"
            else if not (ENumero(r[Quantidade]) and ENumero(r[Valor_Unit]) and ENumero(r[Desconto]))
                 then "Valor não numérico"
            else if r[Valor_Unit] = null or Number.From(r[Quantidade], "pt-BR") <= 0
                 then "Quantidade ou valor inválido"
            else if not List.Contains(Filiais, Chave(r[Filial])) then "Filial sem correspondência"
            else if not List.Contains(Produtos, Chave(r[Produto])) then "Produto não cadastrado"
            else "Linha duplicada",
    ComMotivo = Table.AddColumn(SoRejeitadas, "Motivo", each Diagnostico(_), type text),
    Ordenada = Table.Sort(ComMotivo, {{"Arquivo_Origem", Order.Ascending}, {"Linha_Origem", Order.
    Ascending}})
in
    Ordenada
```

Recurso de exceção	Onde aparece e para quê
`try ... otherwise ...`	**fnData**, **fnHora** e ENumero: transforma um erro previsível em um valor controlado (`null` ou `false`).
`Table.RemoveRowsWithErrors`	**fVendas**, etapa SemErrosDeTipo: tira da base as linhas com erro de conversão.
Manter Erros (`Table.SelectRowsWithErrors`)	Uso para inspeção: mostra só as linhas com erro.
`JoinKind.LeftAnti`	**Rejeitados**: encontra tudo o que foi descartado, por qualquer motivo, sem precisar repetir cada filtro.

Atenção

As regras de `fVendas` e de `Diagnostico` precisam andar juntas. Se você mudar uma regra em `fVendas` (por exemplo, o período), mude também em `Diagnostico`; caso contrário, o motivo registrado fica errado (a **contagem** continua certa, porque vem da anti-junção).

RESULTADO ESPERADO

■ EXEMPLO SIMULADO – não são dados oficiais do projeto

Arquivo	Linha	ID_Cupom	Valor que chama atenção	Motivo
F01	4	F01-000201	cópia da linha 3	Linha duplicada
F01	6	(vazio)	linha toda vazia	Linha em branco
F01	8	F01-000204	Quantidade vazia	Quantidade vazia
F01	9	F01-000205	Data 10/07/2026	Data fora do período
F01	10	F01-000206	Preço `n/d`	Valor não numérico
F01	12	F01-000207	`Torta Salgada␣` (= linha 11)	Linha duplicada
F01	13	(vazio)	cupom ausente	ID_Cupom vazio
F02	5	F02-000103	`Capuccino`	Produto não cadastrado
F02	6	F02-000104	Qtd vazia	Quantidade vazia
F02	7	F02-000105	Qtd 0	Quantidade zero
F02	12	F02-000110	`Z. Sul`	Filial sem correspondência
F02	13	F02-000111	`31/02/2026`	Data inválida

12 linhas, com os valores **originais** (sem tratamento), mais `Arquivo_Origem`, `Linha_Origem` e `Motivo`.

Como conferir

Escolha uma linha de **`Rejeitados`** ao acaso, abra o arquivo bruto indicado em `Arquivo_Origem` e vá à linha `Linha_Origem`: os valores devem ser os mesmos. É a mesma prova de rastreabilidade da Etapa 1.

ERROS COMUNS E CORREÇÕES

Erro	Correção
`Rejeitados` tem 27 linhas (tudo)	Na mesclagem, as colunas foram selecionadas em ordens diferentes nas duas tabelas, ou **`fVendas`** perdeu `Linha_Origem`.
Motivo `Linha duplicada` para uma linha claramente inválida	`Diagnostico` está desatualizado em relação a **`fVendas`** (atenção acima).
`Formula.Firewall` ao atualizar	Refaça a configuração de privacidade (Seção 2.2).
Atualização lenta na base oficial	Confirme o `List.Buffer` nas listas. A anti-junção é rápida; o custo maior é ler os arquivos.

9.2 Consultas de conferência

OBJETIVO

Provar, a cada atualização, que nenhuma linha se perdeu, e resumir os rejeitados por motivo.

PASSOS

Crie as consultas nulas `Conf_Reconciliacao` e `Conf_Motivos` no grupo `06_Conferencia`.

CÓDIGO M EXATO

```
// Conf_Reconciliacao -- a diferença tem de ser zero
let
    Lidas = Table.RowCount(stg_Vendas_Bruto),
    Aceitas = Table.RowCount(fVendas),
    Rejeitadas = Table.RowCount(Rejeitados),
    Resultado = #table(type table [Indicador = text, Linhas = Int64.Type], {
        {"1. Linhas lidas (todas as filiais)", Lidas},
        {"2. Aceitas em fVendas", Aceitas},
        {"3. Registradas em Rejeitados", Rejeitadas},
        {"4. Diferença (1 - 2 - 3): deve ser 0", Lidas - Aceitas - Rejeitadas}})
in
    Resultado

// Conf_Motivos -- quantas linhas por motivo
let
    Agrupada = Table.Group(Rejeitados, {"Motivo"}, {{"Linhas", each Table.RowCount(_), Int64.Type}}),

    Ordenada = Table.Sort(Agrupada, {{"Linhas", Order.Descending}, {"Motivo", Order.Ascending}})
in
    Ordenada
```

RESULTADO ESPERADO

EXEMPLO SIMULADO – não são dados oficiais do projeto

Conf_Reconciliacao	Linhas
1. Linhas lidas	27
2. Aceitas em fVendas	15
3. Registradas em Rejeitados	12
4. Diferença	**0**

Conf_Motivos	Linhas
Linha duplicada	2
Quantidade vazia	2
Data fora do período	1
Data inválida	1
Filial sem correspondência	1
ID_Cupom vazio	1
Linha em branco	1
Produto não cadastrado	1
Quantidade zero	1
Valor não numérico	1

Como conferir

A soma de `Conf_Motivos` (12) é igual à linha 3 de `Conf_Reconciliacao`. Na base oficial, os números mudam, mas a **diferença continua 0**.

10. Atividade 9 – Carga dos dados tratados para o Excel

10.1 Consulta fMetas: despivotar as metas mensais

OBJETIVO

Transformar a planilha de metas, que tem um mês por coluna (formato largo), em uma tabela com uma linha

por filial e mês (formato longo), como exige o modelo da Etapa 3.

O QUE PRECISO TER ANTES DE COMEÇAR

Parâmetro `pArquivoMetas`. O arquivo simulado tem as colunas `Cod_Filial`, `Filial`, `2026-01` ... `2026-06`.

PASSOS

1. Consulta nula **`fMetas`** (grupo `05_Saida`). Cole as etapas `Fonte` e `Planilha` do código abaixo.
2. Remova a coluna `Filial` (o nome vem de **`dFilial`**).
3. Selecione `Cod_Filial` → **Transformar** → **Transformar Colunas em Linhas** → **Transformar Outras Colunas em Linhas**. Renomeie `Atributo` para `AnoMes` e `Valor` para `Meta` (ou edite o código, como abaixo).
4. Ajuste os tipos com localidade e acrescente a coluna `Mes_Inicio` (primeiro dia do mês), útil para relacionar com o calendário na Etapa 3.

CÓDIGO M EXATO

```
let
    Fonte = Excel.Workbook(File.Contents(pArquivoMetas), true),
    Planilha = Table.SelectRows(Fonte, each [Kind] = "Sheet"){0}[Data],
    SemNome = Table.RemoveColumns(Planilha, {"Filial"}),
    // um mês por coluna -> uma linha por (filial, mês)
    Despivotada = Table.UnpivotOtherColumns(SemNome, {"Cod_Filial"}, "AnoMes", "Meta"),
    Tipos = Table.TransformColumnTypes(Despivotada,
              {{"Cod_Filial", type text}, {"AnoMes", type text}, {"Meta", type number}}, "pt-BR"),
    MesInicio = Table.AddColumn(Tipos, "Mes_Inicio", each
                  #date(Number.From(Text.Start([AnoMes], 4)), Number.From(Text.End([AnoMes], 2)), 1),
     type date)
in
    MesInicio
```

RESULTADO ESPERADO

■ EXEMPLO SIMULADO – não são dados oficiais do projeto

`fMetas` com **30 linhas** (5 filiais × 6 meses) e 4 colunas. Exemplo: `F01 | 2026-01 | 62000 | 01/01/2026`. Soma de `Meta`: **1.441.000**. Os valores são fictícios, criados só para o exercício.

Como conferir

Agrupe por `Cod_Filial` (teste, depois exclua): 5 grupos com 6 linhas cada. Se aparecerem 7 linhas por filial, há uma coluna de total na planilha de metas: remova-a antes de despivotar.

ERROS COMUNS E CORREÇÕES

Erro	Correção
Usou **Transformar Colunas em Linhas** (só as selecionadas) com os 6 meses	Funciona hoje, mas um mês novo (2026-07) ficaria de fora. **Outras Colunas** se adapta sozinho.
`AnoMes` veio como data	O cabeçalho estava formatado como data no Excel. Ajuste a planilha de metas para texto `aaaa-mm` ou converta com `Date.ToText`.

10.2 Carregar as consultas em planilhas

OBJETIVO

Levar para o Excel apenas as consultas de saída e de conferência, cada uma em sua planilha, deixando as de apoio como *Somente conexão*.

O QUE PRECISO TER ANTES DE COMEÇAR

Todas as consultas criadas e sem erro (nenhum triângulo amarelo no painel **Consultas**).

PASSOS

1. No Editor: **Página Inicial** → **Fechar e Carregar** → **Fechar e Carregar Para...** → **Apenas Criar Conexão** → **OK**. Tudo volta ao Excel como conexão.
2. No Excel, **Dados** → **Consultas e Conexões**. Para `fVendas`: botão direito → **Carregar Para...** → **Tabela** → **Nova planilha** → **OK**. Deixe **Adicionar estes dados ao Modelo de Dados desmarcado** (o modelo é da Etapa 3).
3. Repita para `Rejeitados`, `dProduto`, `dFilial` e `fMetas`.
4. Renomeie as planilhas com o nome da consulta: `fVendas`, `Rejeitados`, `dProduto`, `dFilial`, `fMetas`. Apague a `Planilha1` vazia.
5. Crie a planilha `Conferencia`. Carregue `Conf_Reconciliacao` em `A1` (**Carregar Para** → **Tabela** → **Planilha existente** `=Conferencia!$A$1`), `Conf_Motivos` em `D1` e `Conf_Colunas` em `G1`.
6. Salve (Ctrl + B).

FÓRMULAS OU VALORES EXATOS

Consulta	Carga	Grupo
`pPastaVendas`, `pArquivoEtapa1`, `pArquivoMetas`	parâmetro	`01_Parametros`
`fnLerPlanilha`, `fnPadronizarColunas`, `fnLerVendas`, `fnData`, `fnHora`, `fnValorOficial`	função	`02_Funcoes`
`cad_Etapa1`, `cad_DeParaFilial`, `cad_ProdutoChave`, `lst_Pagamento`, `lst_Canal`, `ColunasAlvo`, `DeParaColunas`	somente conexão	`03_Cadastros`
`stg_Vendas_Bruto`	somente conexão	`04_Staging`
`fVendas`, `Rejeitados`, `dProduto`, `dFilial`, `fMetas`	**tabela** em planilha própria	`05_Saida`
`Conf_Reconciliacao`, `Conf_Motivos`, `Conf_Colunas`	**tabela** na aba `Conferencia`	`06_Conferencia`

RESULTADO ESPERADO

O que você deve ver na tela

Planilhas `fVendas` (15 linhas), `Rejeitados` (12), `dProduto` (18), `dFilial` (5), `fMetas` (30) e `Conferencia`, cada tabela com o nome da consulta (**Design da Tabela** → **Nome da Tabela**: `fVendas` etc.). No painel **Consultas e Conexões**, cada consulta carregada mostra *N linhas carregadas*; as demais, *Somente conexão*.

Como conferir

1. Na aba `Conferencia`: Diferença **0**; `Conf_Colunas` sem texto na coluna `Colunas_Nao_Mapeadas`.
2. Numa célula livre de `Conferencia`: `=SOMA(fVendas[Valor_Liquido])` → **210**; `=CONT.VALORES(fVendas[ID_Cupom])` → 15. Apague depois.
3. **Teste de atualização (reenvio de arquivo).** Copie `vendas_F01_sp_centro_SIMULADO.xlsx` para a mesma pasta com o nome `vendas_F99_teste_reenvio.xlsx`. No Excel, **Dados** → **Atualizar Tudo**. Esperado: lidas **39**, aceitas **15** (inalterado: as vendas reenviadas são duplicatas), rejeitadas **24**, diferença 0; em `Conf_Motivos`, `Linha duplicada` passa a **9**. Apague o arquivo de teste e atualize de novo: voltam 27/15/12.

ERROS COMUNS E CORREÇÕES

Erro	Correção
Todas as consultas carregaram em planilhas	Para cada consulta de apoio: botão direito → **Carregar Para...** → **Apenas Criar Conexão**. Apague as planilhas que sobrarem.
Editou um valor dentro da tabela `fVendas`	Será apagado na próxima atualização. Correções são feitas na origem (ou no de-para da Etapa 1), nunca na tabela carregada.
Download não concluído / erro numa consulta	Clique no link de erro no painel: o Power Query mostra a linha. Na maioria das vezes é arquivo aberto ou caminho do parâmetro.
Colunas da tabela carregada mudaram de largura e formato	Formate datas e moedas no Excel *depois* da carga e marque, em **Design da Tabela** → **Propriedades**, **Preservar formatação de colunas**.

10.3 Publicar no repositório e revisar a entrega

OBJETIVO

Deixar em `MA2026-entrega-parcial/02-powerquery/` o arquivo, o código M, os dados limpos e as evidências, e conferir item a item.

PASSOS

1. **Scripts M.** Para cada consulta: Editor → selecione a consulta → **Editor Avançado** → `Ctrl`+`A`, `Ctrl`+`C` → cole no Bloco de Notas → salve em `02-powerquery\scripts_m\<NomeDaConsulta>.pq` com **Codificação: UTF-8**. A primeira linha do arquivo deve ser um comentário com o nome: `// fVendas`.
2. **Dados limpos.** Na aba `fVendas`, selecione a tabela inteira → `Ctrl`+`C` → nova pasta de trabalho → **Colar Especial** → **Valores** → **Salvar Como** → **CSV UTF-8 (Delimitado por vírgulas)** → `dados_limpos\fVendas.csv`. Feche sem salvar a pasta temporária.
3. **Evidências.** Capturas (`Win`+`Shift`+`S`) do painel de consultas com os grupos, das **Etapas Aplicadas** de `fVendas` e da aba `Conferencia`, salvas em `evidencias\`.
4. **README.md** com: lista das consultas e o que cada uma faz; valores dos parâmetros; regras de rejeição; decisões pendentes (`Z. Sul`, `App`, `Capuccino`).
5. Faça o *commit* com mensagem descritiva (por exemplo, `Etapa 2: consultas Power Query e dados limpos`). Cada integrante deve ter commits próprios: o histórico é avaliado.

TABELA DE CONFERÊNCIA

Status possíveis: **Não iniciado**, **Em andamento**, **Concluído**, **Pendente de esclarecimento**. Só marque *Concluído* depois de executar a verificação no arquivo.

Item	Resultado esperado	Como verificar	Detalhe / valor no simulado	Status
1	Parâmetros configurados	Painel **Consultas**, grupo `01_Parametros`	3 parâmetros; caminhos abrem no Explorador	
2	Pasta lida inteira	`stg_Vendas_Bruto`	27 linhas, 13 colunas, 2 arquivos	
3	D10 resolvido	`Conf_Colunas`	nenhuma coluna não mapeada	
4	Nulos e duplicatas	Contagem por etapa (Seção 8.1)	22 após nulos; 21 e 15 após as duas passagens	
5	Tipos com localidade	Ícones; **Perfil da Coluna** de `Valor_Unit`	máximo 65; `"8,50"` → 8,5	
6	Textos padronizados	Filtros de `Categoria` e `Pagamento`	3 categorias; pagamentos na grafia oficial	
7	Mesclagens	Linhas antes/depois de expandir	18 → 18; nenhum `Cod_Filial`/`Cod_Produto` nulo	
8	Colunas calculadas	Soma de `Valor_Bruto` e `Valor_Liquido`	212,00 e 210,00	
9	Rejeitados rastreáveis	Sortear 2 linhas e achar no bruto	mesmo cupom e mesmos valores	
10	Reconciliação	Aba `Conferencia`	$27 = 15 + 12$; diferença 0	
11	Atualização funciona	Teste do reenvio (Seção 10.2)	39/15/24 e volta a 27/15/12	
12	Artefatos no repositório	Pasta `02-powerquery`	`.xlsx`, `scripts_m`, `dados_limpos`, `evidencias`, `README.md`	
13	Escopo respeitado	Ler as colunas de `fVendas`	sem relacionamentos, DAX, macro ou `Custo_Total`	

Atenção

A etapa **não está concluída** se algum item estiver em *Não iniciado* ou *Em andamento*, ou se a diferença da reconciliação não for 0. Itens *Pendente de esclarecimento* são aceitos desde que estejam no `README.md` e em `Rejeitados`.

A. Código M completo da consulta fVendas

Copie para o **Editor Avançado** de `fVendas` se quiser conferir ou reconstruir a consulta. As funções e consultas citadas precisam existir com estes nomes.

```
let
    Origem = stg_Vendas_Bruto,

    // ---------- Atividade 3: linhas em branco, nulos e duplicatas (1ª passagem) ----------
    SemLinhasEmBranco = Table.SelectRows(Origem, each not List.IsEmpty(
        List.RemoveMatchingItems(Record.FieldValues(Record.SelectFields(_, ColunasAlvo)), {"", null
    }))),
    SemCupomVazio = Table.SelectRows(SemLinhasEmBranco, each [ID_Cupom] <> null
                                     and Text.Trim(Text.From([ID_Cupom])) <> ""),
    SemQtdVazia = Table.SelectRows(SemCupomVazio, each [Quantidade] <> null and [Quantidade] <> 0),
    DescontoNuloZero = Table.ReplaceValue(SemQtdVazia, null, 0, Replacer.ReplaceValue, {"Desconto"}),

    SemDuplicatasExatas = Table.Distinct(DescontoNuloZero, {"ID_Cupom", "Produto", "Hora"}),

    // ---------- Atividade 4: tipos e localidade pt-BR ----------
    DatasHoras = Table.TransformColumns(SemDuplicatasExatas,
                  {{"Data", fnData, type nullable date}, {"Hora", fnHora, type nullable time}}),
    SemDataInvalida = Table.SelectRows(DatasHoras, each [Data] <> null),
    NoPeriodo = Table.SelectRows(SemDataInvalida, each [Data] >= #date(2026, 1, 1)
```

```
                                                and [Data] <= #date(2026, 6, 30)),
    SemHoraInvalida = Table.SelectRows(NoPeriodo, each [Hora] <> null),
    TiposPtBR = Table.TransformColumnTypes(SemHoraInvalida,
                  {{"ID_Cupom", type text}, {"Quantidade", Int64.Type},
                   {"Valor_Unit", type number}, {"Desconto", type number}}, "pt-BR"),
    SemErrosDeTipo = Table.RemoveRowsWithErrors(TiposPtBR, {"Quantidade", "Valor_Unit", "Desconto"}),

    QtdEValorValidos = Table.SelectRows(SemErrosDeTipo, each [Quantidade] > 0 and [Valor_Unit] <>
    null),

    // ---------- Atividade 5: textos ----------
    TextosAparados = Table.TransformColumns(QtdEValorValidos,
        {{"Filial", Text.Trim, type text}, {"Produto", Text.Trim, type text},
         {"Categoria", Text.Trim, type text}, {"Pagamento", Text.Trim, type text},
         {"Canal", Text.Trim, type text}}),
    PrimeiraMaiuscula = Table.TransformColumns(TextosAparados,
        {{"Produto", Text.Proper, type text}, {"Categoria", Text.Proper, type text}}),
    PagamentoOficial = Table.TransformColumns(PrimeiraMaiuscula,
        {{"Pagamento", each fnValorOficial(_, lst_Pagamento), type text}}),
    CanalOficial = Table.TransformColumns(PagamentoOficial,
        {{"Canal", each fnValorOficial(_, lst_Canal), type text}}),

    // ---------- Atividade 6: mesclagens e duplicatas (2ª passagem) ----------
    ChaveFilial = Table.AddColumn(CanalOficial, "Chave_Filial", each Text.Upper([Filial]), type text
    ),
    MesclaFilial = Table.NestedJoin(ChaveFilial, {"Chave_Filial"}, cad_DeParaFilial, {"Chave_Filial"
    },
                                    "DePara", JoinKind.LeftOuter),
    ExpandeFilial = Table.ExpandTableColumn(MesclaFilial, "DePara", {"Cod_Filial"}, {"Cod_Filial"}),
    ComFilial = Table.SelectRows(ExpandeFilial, each [Cod_Filial] <> null),
    ChaveProduto = Table.AddColumn(ComFilial, "Chave_Produto", each Text.Upper([Produto]), type text
    ),
    MesclaProduto = Table.NestedJoin(ChaveProduto, {"Chave_Produto"}, cad_ProdutoChave, {"
    Chave_Produto"},
                                     "Cadastro", JoinKind.LeftOuter),
    ExpandeProduto = Table.ExpandTableColumn(MesclaProduto, "Cadastro", {"Cod_Produto"}, {"
    Cod_Produto"}),
    ComProduto = Table.SelectRows(ExpandeProduto, each [Cod_Produto] <> null),
    OrdemOriginal = Table.Buffer(Table.Sort(ComProduto,
                      {{"Arquivo_Origem", Order.Ascending}, {"Linha_Origem", Order.Ascending}})),
    SemDuplicatas = Table.Distinct(OrdemOriginal, {"ID_Cupom", "Cod_Produto", "Hora"}),

    // ---------- Atividade 7: colunas personalizadas e formato final ----------
    ValorBruto = Table.AddColumn(SemDuplicatas, "Valor_Bruto",
                   each Number.Round([Quantidade] * [Valor_Unit], 2), type number),
    ValorLiquido = Table.AddColumn(ValorBruto, "Valor_Liquido",
                   each Number.Round([Valor_Bruto] - [Desconto], 2), type number),
    FormatoFinal = Table.SelectColumns(ValorLiquido,
        {"ID_Cupom", "Data", "Hora", "Cod_Filial", "Cod_Produto", "Quantidade", "Valor_Unit",
         "Desconto", "Valor_Bruto", "Valor_Liquido", "Pagamento", "Canal",
         "Arquivo_Origem", "Linha_Origem"})
in
    FormatoFinal
```

COMPARAÇÃO COM O TRECHO DA ESPECIFICAÇÃO (SEÇÃO 6.2)

Trecho do Projeto Modelo	Neste guia
`Excel.Workbook([Content], true){0}[Data]`	**`fnLerPlanilha`** filtra `[Kind] = "Sheet"` antes do `{0}`: o item 0 pode ser uma tabela ou um nome definido, não a aba.
`Table.Combine(Conteudo[Dados])`	`Table.ExpandTableColumn`: mesmo resultado, mas guarda `Arquivo_Origem`. A padronização (D10) acontece antes, em **`fnLerVendas`**.
`Table.TransformColumnTypes(..., "pt-BR")` para `Data`	Datas vão por **`fnData`**, porque a coluna mistura formatos (D1) e tem data inexistente.
`Table.SelectRows(...[Quantidade] <> null and [Quantidade] > 0)`	Mesma regra, em duas etapas (`SemQtdVazia` e `QtdEValorValidos`).
`Table.Distinct(..., {"ID_Cupom", "Cod_Produto", "Hora"})`	Mantido (`SemDuplicatas`), precedido de uma passagem com o nome do produto e de `Table.Buffer`.
`Valor_Bruto`	Mantido, com arredondamento; acrescentado `Valor_Liquido` (Seção 4.2).

B. Funções e trechos complementares

FNLIMPARTEXTO – EQUIVALENTE EXATO DO ARRUMAR (ESPAÇOS INTERNOS REPETIDOS)

```
(texto as nullable text) as nullable text =>
    if texto = null then null
    else Text.Combine(List.Select(Text.Split(Text.Trim(texto), " "), each _ <> ""), " ")
```

Para usar, troque `Text.Trim` por `fnLimparTexto` na etapa `TextosAparados` e nas funções `Chave` de **`Rejeitados`** e das consultas `cad_`.

PERÍODO EM PARÂMETROS (PARA NÃO REPETIR DATAS NO CÓDIGO)

```
// parâmetros pDataInicio e pDataFim (Tipo: Data), usados em fVendas e em Rejeitados
NoPeriodo = Table.SelectRows(SemDataInvalida, each [Data] >= pDataInicio and [Data] <= pDataFim),
```

TIPO MOEDA (DECIMAL FIXO) EM VEZ DE DECIMAL

```
{"Valor_Unit", Currency.Type}   // 4 casas decimais fixas; evita resíduos de arredondamento
```

C. Filial que envia CSV em vez de planilha

Se uma filial mandar `.csv` (separado por ponto e vírgula, como o Excel em português salva), acrescente uma função de leitura e combine as duas origens.

```
// fnLerVendasCsv -- mesmo resultado de fnLerVendas, para arquivos .csv
(conteudo as binary) as table =>
let
    Csv = Csv.Document(conteudo, [Delimiter = ";", Encoding = 65001, QuoteStyle = QuoteStyle.Csv]),
    ComCabecalho = Table.PromoteHeaders(Csv, [PromoteAllScalars = true]),
    Padronizada = fnPadronizarColunas(ComCabecalho),
    ComLinha = Table.AddIndexColumn(Padronizada, "Linha_Origem", 2, 1, Int64.Type)
in
    ComLinha

// em stg_Vendas_Bruto, troque o filtro de extensão e a etapa ComDados:
    SoPlanilhas = Table.SelectRows(Origem, each List.Contains({".xlsx", ".csv"}, Text.Lower([
    Extension]))
```

```
                                        and not Text.StartsWith([Name], "~$")),
    ComDados = Table.AddColumn(SoPlanilhas, "Dados", each
                if Text.Lower([Extension]) = ".csv" then fnLerVendasCsv([Content]) else fnLerVendas
    ([Content])),
```

No CSV tudo chega como texto: `"2"`, `"6,50"`, `"03/03/2026"`. As Atividades 4 e 5 já tratam isso (**fnData**, **fnHora** e conversão com `"pt-BR"`). O teste `[Quantidade] <> 0` da Atividade 3 não pega o texto `"0"`, mas `QtdEValorValidos` pega depois da conversão. `Encoding = 65001` é UTF-8; se os acentos vierem quebrados, use `1252` (Windows).

D. Gabarito completo do exemplo simulado

EXEMPLO SIMULADO – não são dados oficiais do projeto

Conteúdo esperado de **fVendas** com os dois arquivos simulados. **Não vale para a base oficial.**

Arq.	Lin.	ID_Cupom	Data	Hora	Fil.	Prod.	Qtd	V.Unit	Desc.	Bruto	Líq.	Pagamento	Canal
F01	2	F01-000201	02/01/2026	07:10	F01	P01	1	6,50	0	6,50	6,50	Pix	Balcão
F01	3	F01-000201	02/01/2026	07:10	F01	P13	2	5,50	0	11,00	11,00	Pix	Balcão
F01	5	F01-000202	15/01/2026	08:30	F01	P04	1	10,50	0	10,50	10,50	Débito	Balcão
F01	7	F01-000203	20/01/2026	12:15	F01	P05	1	12,00	1	12,00	11,00	Crédito	Delivery
F01	11	F01-000207	12/02/2026	10:00	F01	P18	1	13,50	0	13,50	13,50	Vale-refeição	Delivery
F02	2	F02-000101	03/03/2026	07:45	F02	P01	2	6,50	0	13,00	13,00	Pix	Balcão
F02	3	F02-000101	03/03/2026	07:45	F02	P13	2	5,50	0	11,00	11,00	Pix	Balcão
F02	4	F02-000102	03/03/2026	09:10	F02	P03	1	9,00	0	9,00	9,00	Débito	Balcão
F02	8	F02-000106	05/03/2026	08:15	F02	P01	1	65,00	0	65,00	65,00	Pix	Balcão
F02	9	F02-000107	05/03/2026	10:20	F02	P14	1	8,50	0	8,50	8,50	Vale-refeição	Balcão
F02	10	F02-000108	06/03/2026	13:00	F02	P15	1	9,00	0	9,00	9,00	Débito	Delivery
F02	11	F02-000109	06/03/2026	05:30	F02	P02	1	5,00	0	5,00	5,00	Pix	Balcão
F02	14	F02-000112	08/03/2026	19:20	F02	P06	2	11,00	1	22,00	21,00	Pix	App
F02	15	F02-000113	08/03/2026	14:10	F02	P16	1	8,50	0	8,50	8,50	Crédito	Balcão
F02	16	F02-000114	09/03/2026	07:05	F02	P11	1	7,50	0	7,50	7,50	Pix	Balcão
		15 linhas					**19**		**2**	**212,00**	**210,00**	13 cupons distintos	

Da auditoria da Etapa 1 (F02) para a Etapa 2:

Linha	Problema registrado na Etapa 1	Situação na Etapa 2
5	Produto não cadastrado (`Capuccino`)	Em **Rejeitados** até a confirmação
6, 7	Quantidade vazia / zero	Em **Rejeitados** (D5 tratado)
8, 10	Preço com desvio $>$ 20%	Aceitas em **fVendas**; continuam pendentes na auditoria (D9)
11	Hora antes das 6h	Aceita; continua pendente na auditoria
12	Filial sem correspondência (`Z. Sul`)	Em **Rejeitados**; volta sozinha se o de-para for atualizado
13	Data inválida; pagamento fora da lista	Em **Rejeitados** pela data
14	Canal fora da lista (`App`)	Aceita com `Canal` = `App`; pendente
15	Valor unitário não numérico (`"8,50"`)	**Resolvido**: 8,5 numérico (D6)
16	Categoria divergente (`Chá Gelado`)	**Resolvido**: **fVendas** não carrega categoria; vale a de **dProduto** (`Frio`)

E. Quando a base oficial for liberada: o que substituir

1. **Retire** os arquivos `_SIMULADO` de `dados_brutos\vendas` (mova para outra pasta) e copie as cinco planilhas oficiais. Marque-as como somente leitura.
2. Troque `pArquivoMetas` para `C:\CafeData\dados_brutos\metas_mensais.xlsx`. Se as metas oficiais já vierem em formato longo (uma linha por filial e mês), pule o despivotamento e ajuste os nomes de colunas em `fMetas`.
3. Refaça o diagnóstico da Seção 3.1 para as cinco filiais e acrescente ao `DeParaColunas` **todos** os nomes encontrados. Confira `Conf_Colunas`.
4. Na Etapa 1, acrescente ao de-para de filiais **somente** grafias confirmadas, salve e feche o arquivo.
5. **Dados** → **Atualizar Tudo**. Os números do simulado deixam de valer; o que vale é: `Conf_Reconciliacao` com diferença 0, `Conf_Colunas` vazia, e cada motivo de `Conf_Motivos` explicado no `README.md`.
6. A base oficial tem cerca de 30 mil linhas: no Editor, ative **Criação de perfil de coluna com base no conjunto de dados inteiro** antes de conferir qualidade e distribuição.

F. Transposição para o tema da equipe

O que se reaproveita é o **método**, não os nomes (Seção 8 da especificação). Para o tema sorteado, a equipe deve produzir:

No CaféData	No tema da equipe
`ColunasAlvo` (11 colunas de venda)	As colunas do dicionário de dados do próprio tema (Etapa 1).
`DeParaColunas`	Os nomes divergentes encontrados nas fontes do tema (anos, órgãos, sistemas diferentes).
Chave `ID_Cupom` + `Cod_Produto` + `Hora`	A chave que identifica **um** evento do tema (viagem, vaga, reserva, notificação, item de pedido).
De-para de filiais	De-para da dimensão com grafias variáveis (estação, cidade, unidade de saúde, categoria).
Período 01/01 a 30/06/2026	O recorte temporal declarado na ficha do tema.
`fMetas` despivotada	Qualquer fonte em formato largo (um ano ou mês por coluna) do tema.
`Rejeitados` + `Conf_Reconciliacao`	**Obrigatórios em qualquer tema**: são a prova de que nenhuma linha se perdeu.

O arquivo de entrega passa a se chamar `<Tema>_02_PowerQuery.xlsx` (por exemplo, `Mobilidade_02_PowerQuery.xlsx`), sempre em `MA2026-entrega-parcial/02-powerquery/`.